



Arduino Workshop

(ESP32 Advanced Level)

ดร. วาธิส ลีลาภัทร

ภาควิชาวิศวกรรมคอมพิวเตอร์ คณะวิศวกรรมศาสตร์

มหาวิทยาลัยขอนแก่น

Agenda

Content

<https://github.com/watisleelapatra/ESP32-Advance>

- ESP32 Wi-Fi Communications
- ESP32 Simple Web Server
- Internet of Things (IoT) using Blynk Platform
- Hardware interrupt
- LCD Custom font

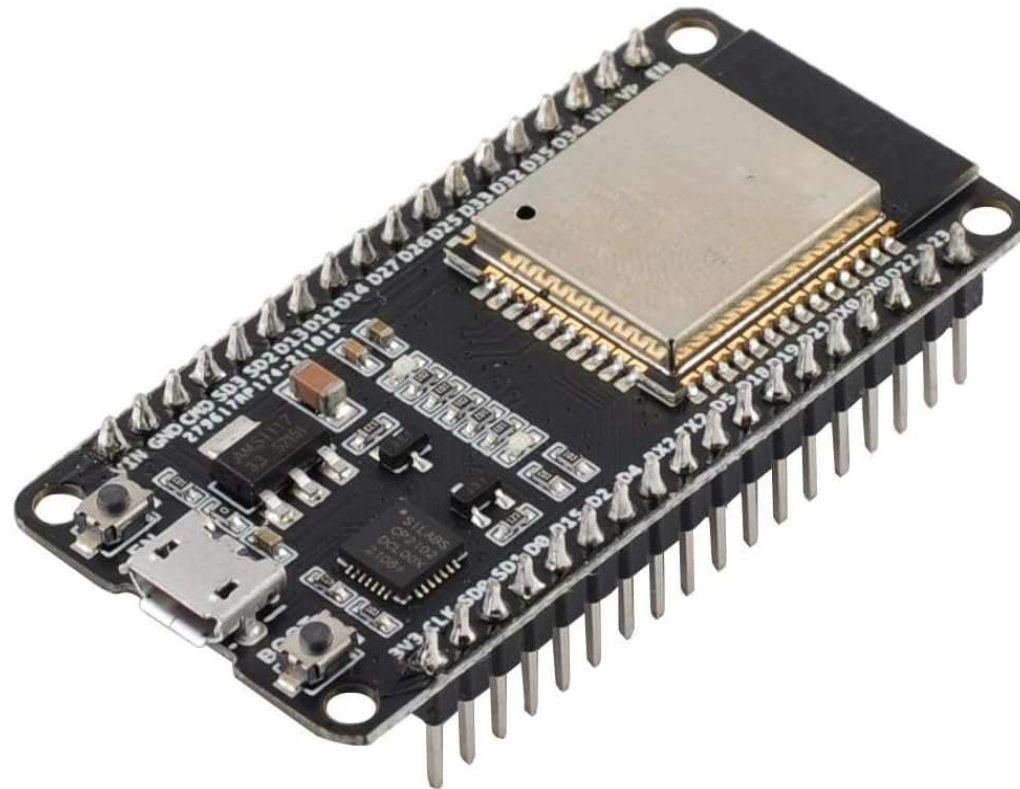
Prerequisite:

1. Basic C/C++ programming language (Arduino IDE)
2. Basic and intermediate ESP32 courses
3. Electronics (not required, but will be advantageous)

ESP32 DEVKIT

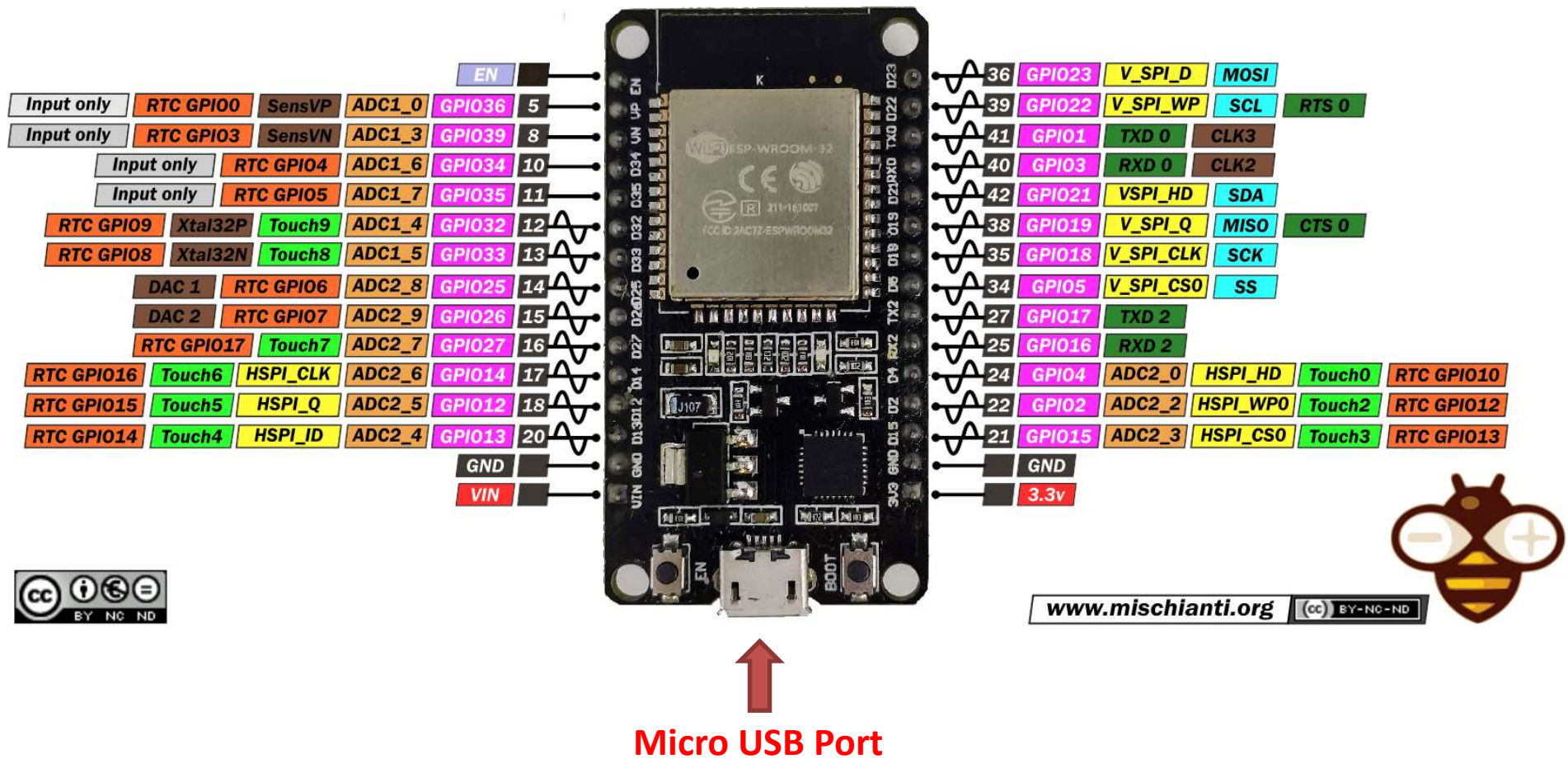
ESP-WROOM-32 DevKit

- utilizes ESP32 chip, 240MHz, RAM 512KB, 4MB flash memory
- Wi-Fi 2.4GHz / Bluetooth



ESP32 DEVKIT Pinout

ESP32 DEV KIT V1 PINOUT



www.mischianti.org



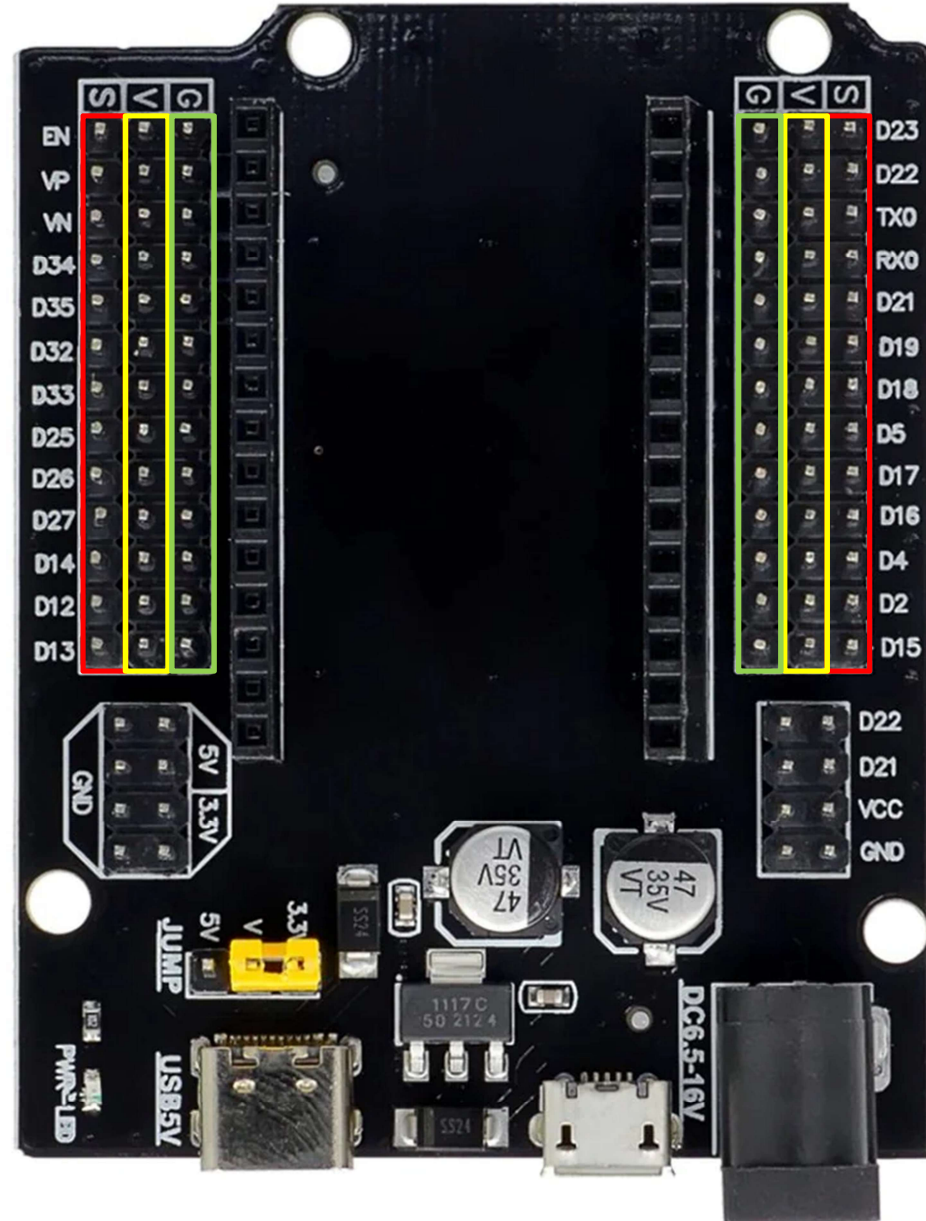
ESP32 Extension Board

- Pins extension
- External power supply

S (Signal) Pins

V (3V Power supply) Pins

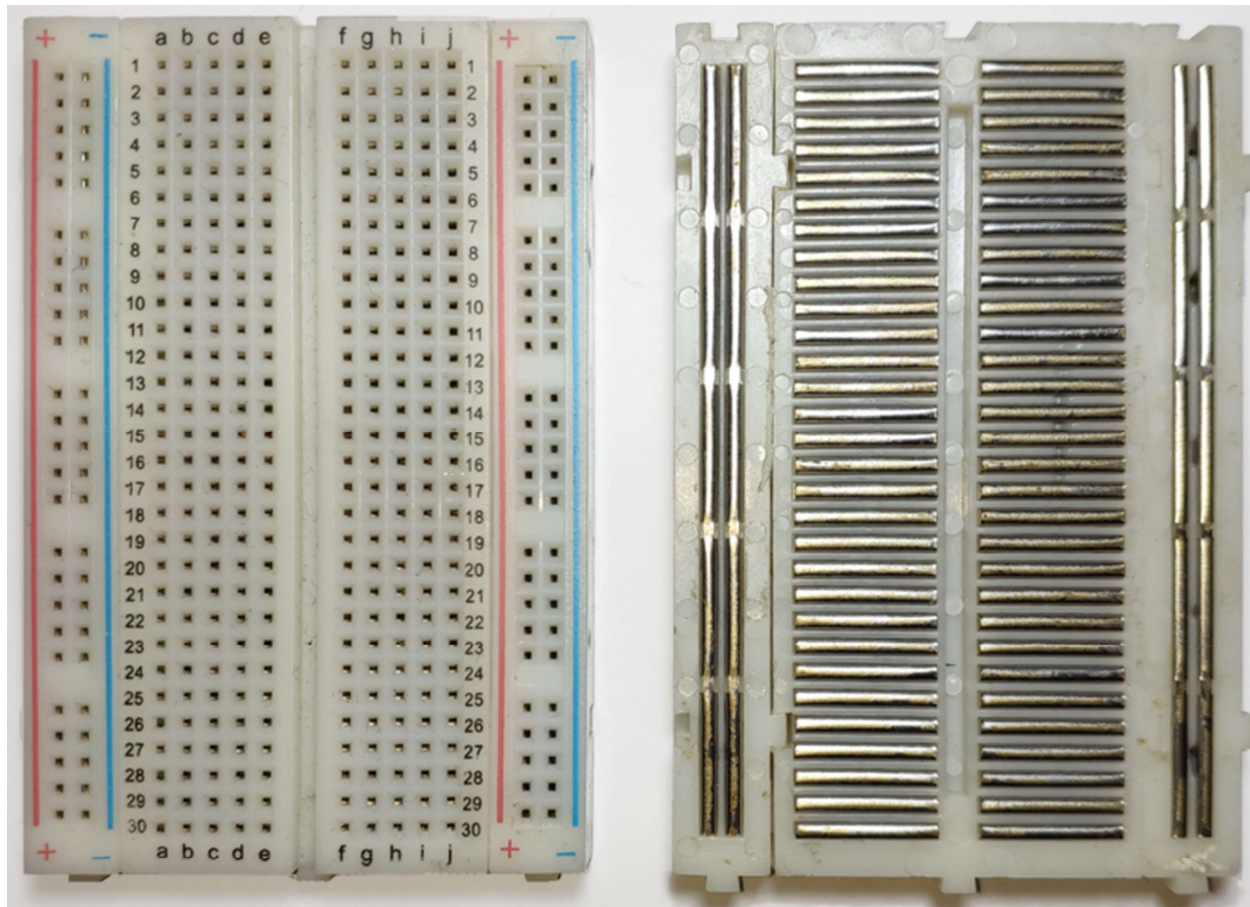
G (Ground) Pins



Circuit Construction

Prototyping board

- Pre-connected conductor stripes
- Horizontal stripes on 2 top rows and 2 bottom rows
- Vertical stripes in the middle



Arduino Reference Sheet

<https://kku.world/e664b0>

Arduino Programming Cheat Sheet

Primary source: Arduino Language Reference
<https://www.arduino.cc/reference/en/>

Structure & Flow

Basic Program Structure

```
void setup() {  
  // Runs once when sketch starts  
}  
  
void loop() {  
  // Runs repeatedly  
}
```

Control Structures

```
if (x < 5) { ... } else { ... }  
while (x < 5) { ... }  
for (int i = 0; i < 10; i++) { ... }  
break; // Exit a loop immediately  
continue; // Go to next iteration  
switch (var) {  
  case 1:  
    ...  
  break;  
  case 2:  
    ...  
  break;  
  default:  
    ...  
}  
return x; // x must match return type  
return; // For void return type
```

Function Definitions

```
<ret. type> <name>(<params>) { ... }  
e.g. int double(int x) {return x*2;}
```

Operators

General Operators

= assignment
+ add - subtract
* multiply / divide
% modulo
== equal to != not equal to
< less than > greater than
<= less than or equal to
/> greater than or equal to
&& and || or
! not

Compound Operators

++ increment
-- decrement
+= compound addition
-= compound subtraction
*= compound multiplication
/= compound division
&= compound bitwise and
|= compound bitwise or

Bitwise Operators

& bitwise and | bitwise or
^ bitwise xor ~ bitwise not
<< shift left >> shift right

Pointer Access

& reference: get a pointer
* dereference: follow a pointer

Built-in Functions

Pin Input/Output

Digital I/O - pins 0-13 A0-A5
pinMode(pin,
(INPUT|OUTPUT|INPUT_PULLUP))
int digitalRead(pin)
digitalWrite(pin, (HIGH|LOW))

Analog In - pins A0-A5

int analogRead(pin)
analogReference(
(DEFAULT|INTERNAL|EXTERNAL))

PWM Out - pins 3 5 6 9 10 11

analogWrite(pin, value) // 0-255

Advanced I/O

tone(pin, freq_Hz, [duration_msec])
noTone(pin)
shiftOut(dataPin, clockPin,
(MSBFIRST|LSBFIRST), value)
shiftIn(dataPin, clockPin,
(MSBFIRST|LSBFIRST))
unsigned long pulseIn(pin,
(HIGH|LOW), [timeout_usec])

Time

unsigned long millis() // Overflows at 50 days
unsigned long micros() // Overflows at 70 minutes
delay(msec)
delayMicroseconds(usec)

Math

min(x, y) max(x, y) abs(x)
sin(rad) cos(rad) tan(rad)
sqrt(x) pow(base, exponent)
constrain(x, minval, maxval)
map(val, fromL, fromH, toL, toH)

Random Numbers

randomSeed(seed) // long or int
long random(max) // 0 to max-1
long random(min, max)

Bits and Bytes

lowByte(x) highByte(x)
bitRead(x, bitn)
bitWrite(x, bitn, bit)
bitSet(x, bitn)
bitClear(x, bitn)
bit(bitn) // bitn: 0=LSB 7=MSB

Type Conversions

char(val) byte(val)
int(val) word(val)
long(val) float(val)

External Interrupts

attachInterrupt(interrupt, func,
(LOW|CHANGE|RISING|FALLING))
detachInterrupt(interrupt)
interrupts()
noInterrupts()

Libraries

Serial - comm. with PC or via RX/TX

begin(long speed) // Up to 115200
end()
int available() // #bytes available
int read() // -1 if none available
int peek() // Read w/o removing
Flush()
print(data) println(data)
write(byte) write(char * string)
write(byte * data, size)
SerialEvent() // Called if data rdy

SoftwareSerial.h - comm. on any pin

SoftwareSerial(rxPin, txPin)
begin(long speed) // Up to 115200
listen() // Only I can listen
isListening() // at a time.
read, peek, print, println, write
// Equivalent to Serial library

EEPROM.h - access non-volatile memory

byte read(addr)
write(addr, byte)
EEPROM[index] // Access as array

Servo.h - control servo motors

attach(pin, [min_usec, max_usec])
write(angle) // 0 to 180
writeMicroseconds(us)
// 1000-2000; 1500 is midpoint
int read() // 0 to 180
bool attached()
detach()

Wire.h - I²c communication

begin() // Join a master
begin(addr) // Join a slave @ addr
requestFrom(address, count)
beginTransmission(addr) // Step 1
send(byte) // Step 2
send(char * string)
send(byte * data, size)
endTransmission() // Step 3
int available() // #bytes available
byte receive() // Get next byte
onReceive(handler)
onRequest(handler)

Variables, Arrays, and Data

Data Types

bool true | false
char -128 - 127, 'a' 's' etc.
unsigned char 0 - 255
byte 0 - 255
int -32768 - 32767
unsigned int 0 - 65535
word 0 - 65535
long -2147483648 - 2147483647
unsigned long 0 - 4294967295
float -3.4028e+38 - 3.4028e+38
double currently same as float
void return type: no return value

Strings

```
char str1[8] =  
{ 'A', 'r', 'd', 'u', 'i', 'n', 'o', '\0' };  
// Includes \0 null termination  
char str2[8] =  
{ 'A', 'r', 'd', 'u', 'i', 'n', 'o' };  
// Compiler adds null termination  
char str3[] = "Arduino";  
char str4[8] = "Arduino";
```

Numeric Constants

123 decimal
0b01111011 binary
0173 octal - base 8
0x7B hexadecimal - base 16
123U force unsigned
123L force long
123UL force unsigned long
123.0 force floating point
1.23e6 1.23*10⁶ = 1230000

Qualifiers

static persists between calls
volatile in RAM (nice for ISR)
const read-only
PROGMEM in flash

Arrays

```
byte myPins[] = { 2, 4, 8, 3, 6 };  
int myInts[6]; // Array of 6 ints  
myInts[0] = 42; // Assigning first  
// index of myInts  
myInts[6] = 12; // ERROR! Indexes  
// are 0 though 5
```



CC BY SA by Mark Liffiton
version: 2024-02-14

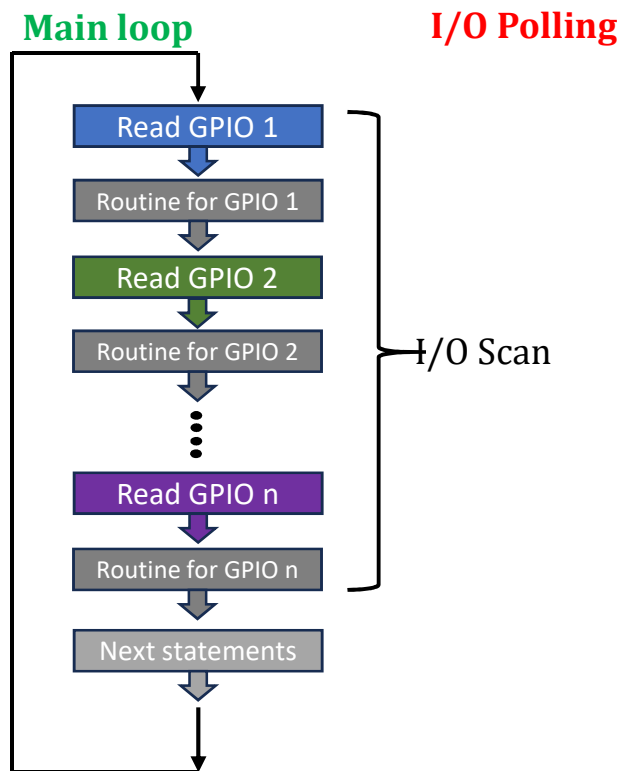
source: <https://github.com/liffiton/Arduino-Cheat-Sheet/>

Adapted from:

- Original: Gavin Smith
- SVG version: Frederic Dufourg
- Arduino board drawing: Fritzing.org

Hardware Interrupt

- Interrupt is a mechanism in microcontroller to handle event triggered by software or internal/external hardware
- Interrupt can reduce processor load significantly when monitoring signal from hardware



Advantages:

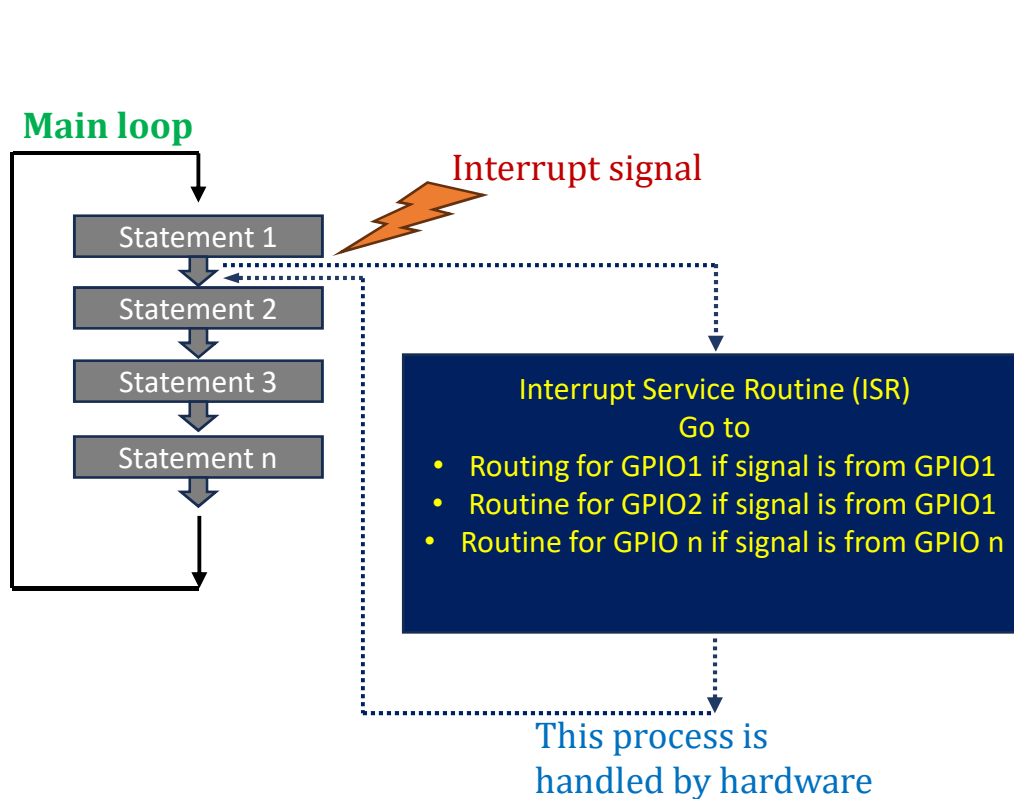
- Implemented in software only

Disadvantages:

- Use more CPU time to execute
- Slow response
- Cannot be used during sleep mode

Hardware Interrupt

- Interrupt is a mechanism in microcontroller to handle event triggered by software or internal/external hardware
- Interrupt can reduce processor load significantly when monitoring signal from hardware



Interrupt

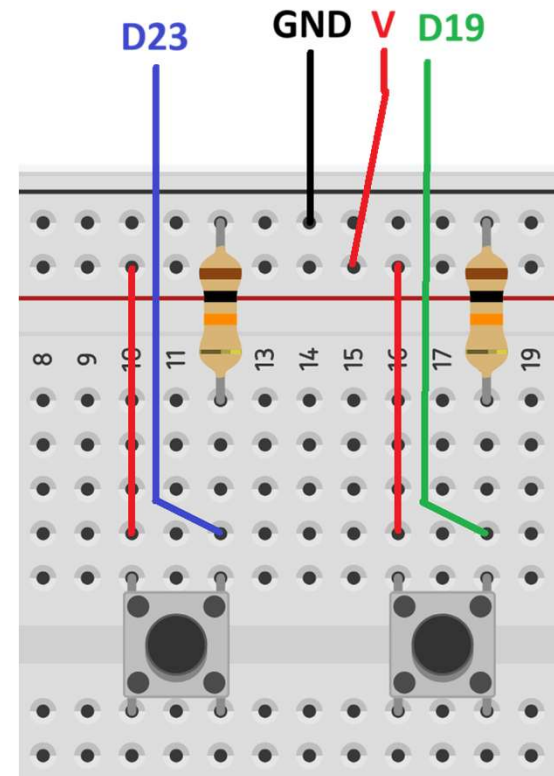
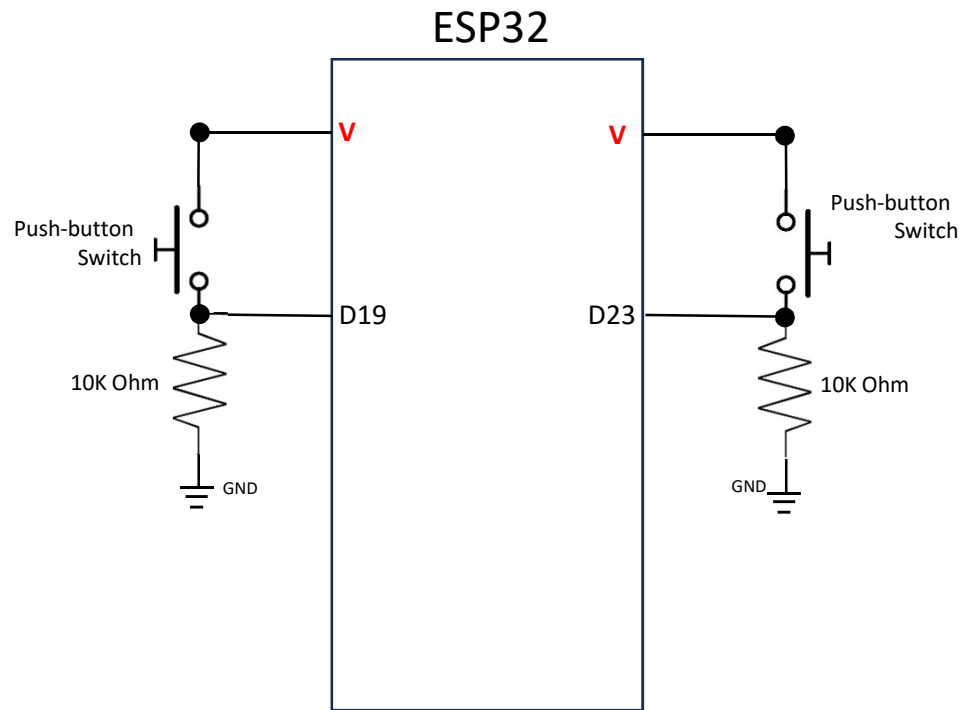
Advantages:

- Fast response
- Free CPU for other tasks
- Can be used during sleep mode

Disadvantages:

- Require hardware support

Hardware Interrupt



Hardware Interrupt

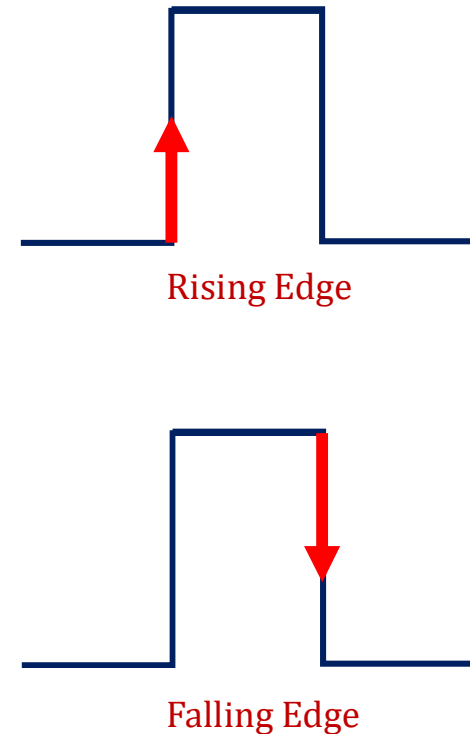
- I/O polling example: [BasicTwoButton.ino](#)
Open serial monitor to see result

```
#define Up 17    //Increase value
#define Down 16 //Decrease value
int Counter = 0;
void setup() {
    Serial.begin(9600);
    pinMode(Up, OUTPUT);
    pinMode(Down, OUTPUT);
}
void loop() {
    Serial.println(Counter);
    if(digitalRead(Up) == 1)
        Counter++;
    if(digitalRead(Down) == 1)
        Counter--;
    delay(100);
}
```

Hardware Interrupt

- I/O polling example: [InterruptTwoButton.ino](#)
Open serial monitor to see result

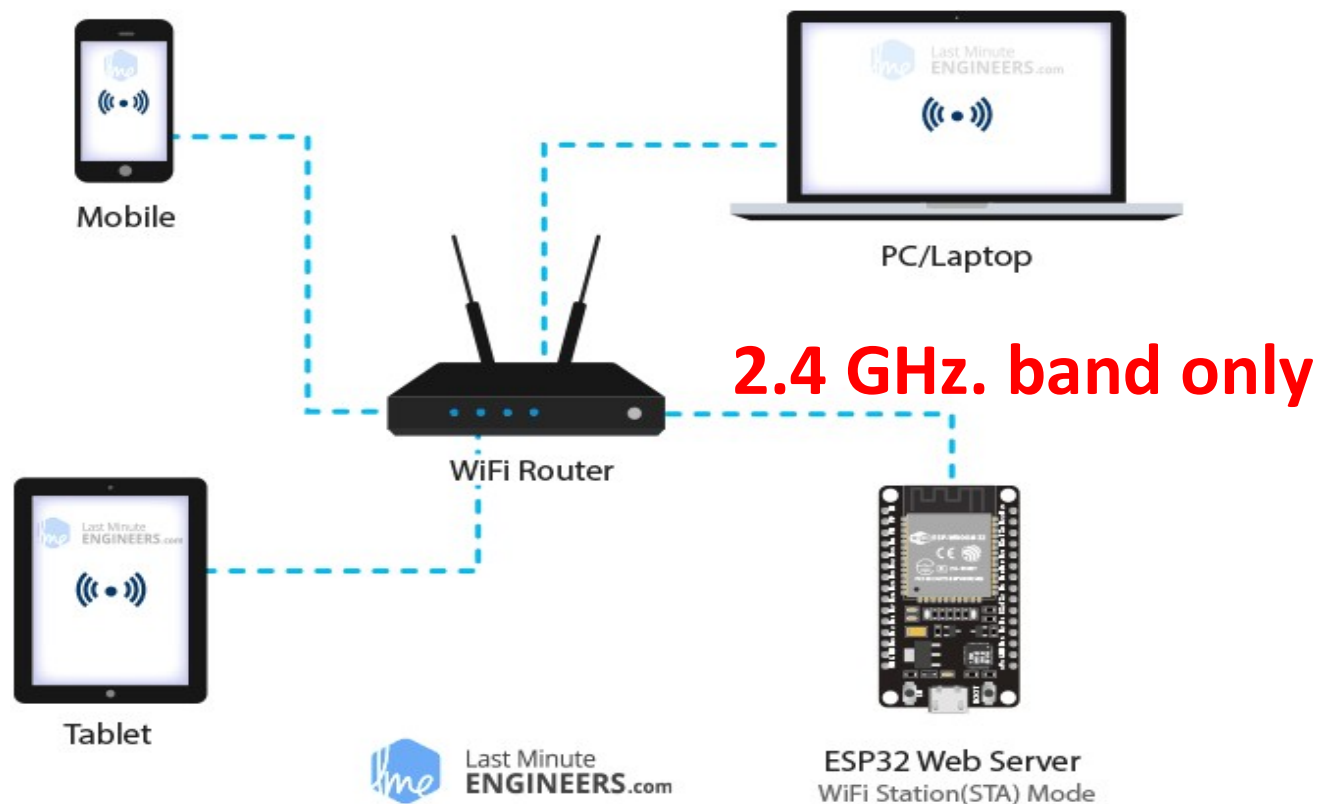
```
#define Up 17    //Increase value
#define Down 16 //Decrease value
int Counter = 0;
void IRAM_ATTR Up_isr() {
    Counter++;
}
void IRAM_ATTR Down_isr() {
    Counter--;
}
void setup() {
    Serial.begin(9600);
    pinMode(Up, OUTPUT);
    pinMode(Down, OUTPUT);
    attachInterrupt(Up, Up_isr, RISING);
    attachInterrupt(Down, Down_isr, RISING);
}
void loop() {
    Serial.println(Counter);
    delay(100);
}
```



ESP32 Wi-Fi Operating Modes

ESP32 can operate in 2 different modes:

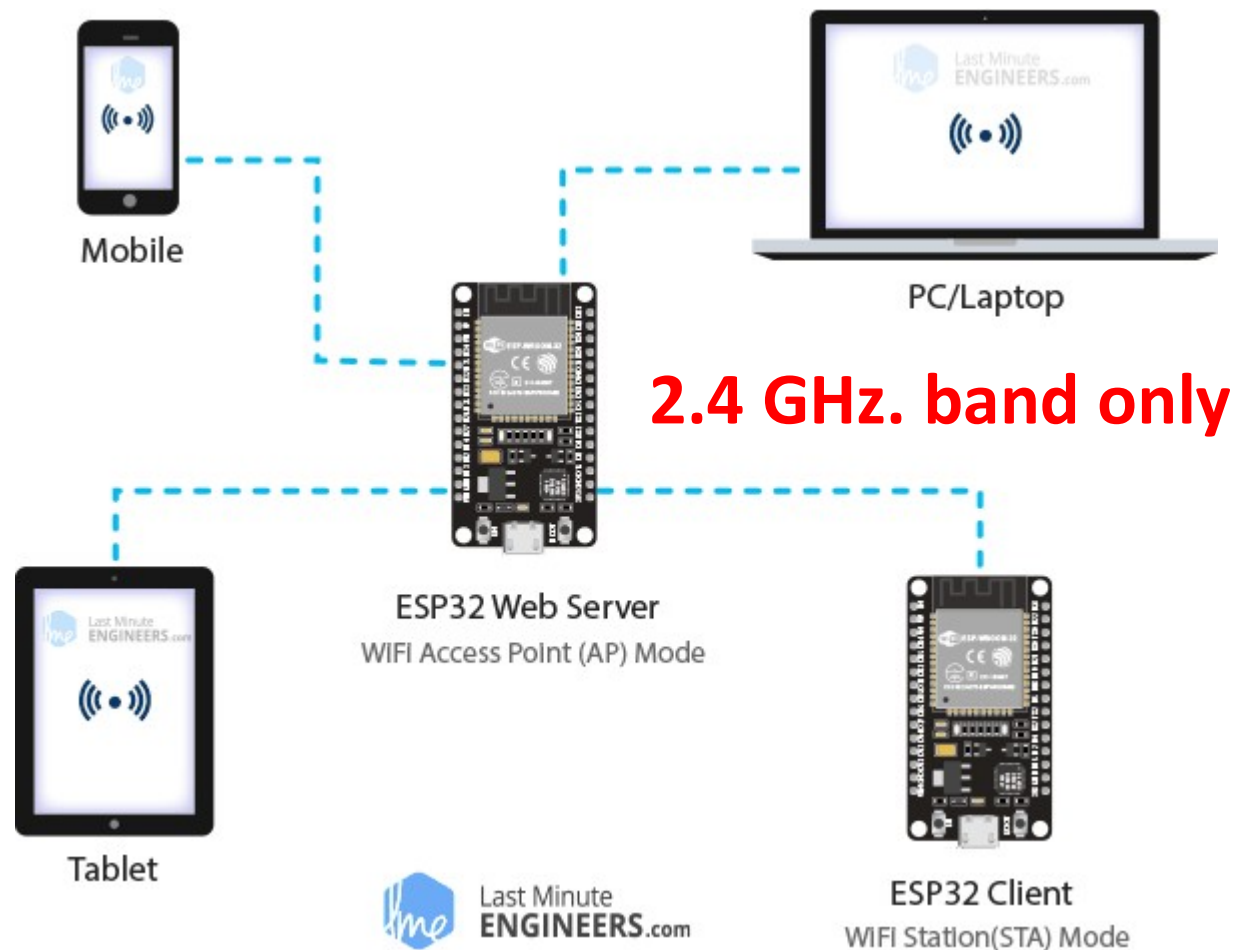
- Station mode
- Soft Access Point mode



ESP32 Wi-Fi Operating Modes

ESP32 can operate in three different modes:

- Station mode
- Soft Access Point mode



Example 1 : Wi-Fi Scanner

This example scans available Wi-Fi access points and displays AP information.

Go to *File* → *Examples* → *WiFi* → ***WiFiScan***

```
void setup() {  
  ...  
  WiFi.mode(WIFI_STA); // Set WiFi to station mode  
  WiFi.disconnect();   // Disconnect from AP if it was previously connected  
  ...
```

Example 1 : Wi-Fi Scanner

// WiFi.scanNetworks will return the number of networks found

```
int n = WiFi.scanNetworks();
```

```
Serial.println("scan done");
```

```
if (n == 0) {
```

```
    Serial.println("no networks found");
```

```
} else {
```

```
    Serial.print(n);
```

```
    Serial.println(" networks found");
```

```
    for (int i = 0; i < n; ++i) {
```

```
        // Print SSID and RSSI (Received Signal Strength Indicator) for each network found
```

```
        Serial.print(i + 1);
```

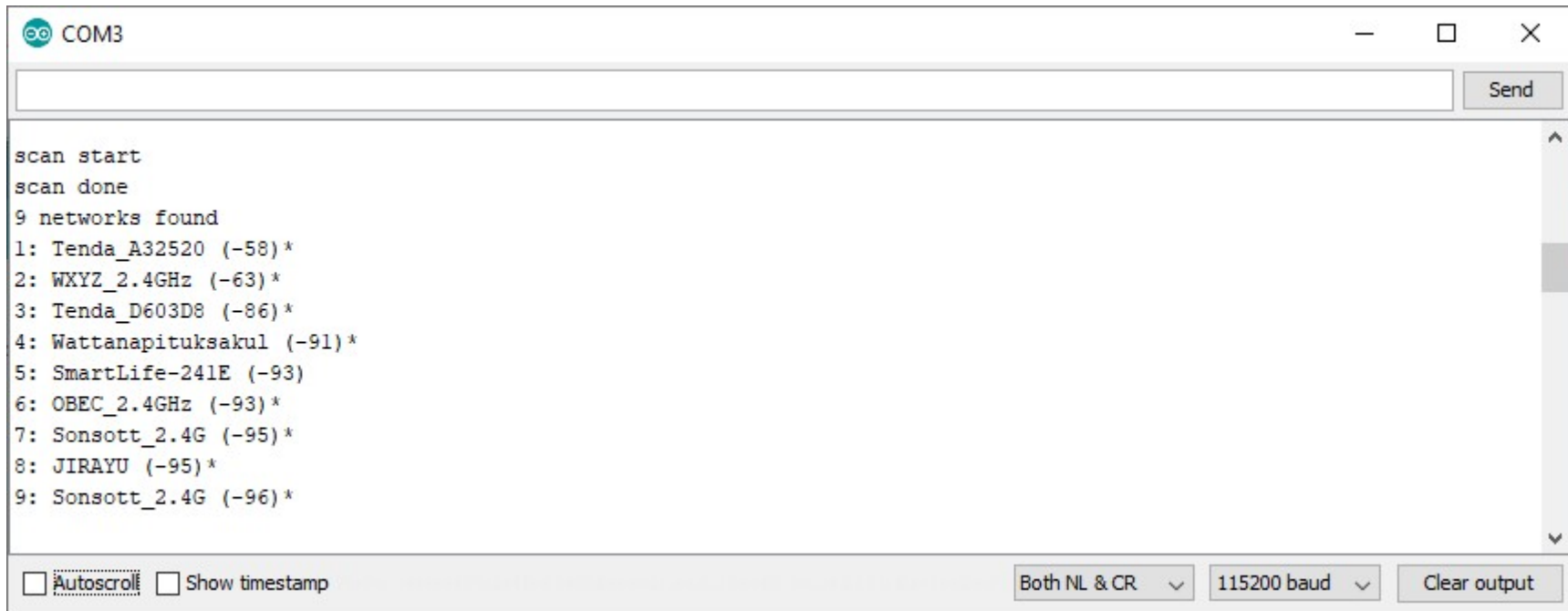
```
        Serial.print(": ");
```

```
        Serial.print(WiFi.SSID(i)); //WiFi.SSID() → access point name
```

```
        Serial.print(" (");
```

```
        Serial.print(WiFi.RSSI(i)); //WiFi.RSSI() → access point signal
```

Example 1 : Wi-Fi Scanner



The screenshot shows a terminal window titled 'COM3' with a 'Send' button. The output text is as follows:

```
scan start
scan done
9 networks found
1: Tenda_A32520 (-58)*
2: WXYZ_2.4GHz (-63)*
3: Tenda_D603D8 (-86)*
4: Wattanapituksakul (-91)*
5: SmartLife-241E (-93)
6: OBEC_2.4GHz (-93)*
7: Sonsott_2.4G (-95)*
8: JIRAYU (-95)*
9: Sonsott_2.4G (-96)*
```

At the bottom of the window, there are checkboxes for 'Autoscroll' and 'Show timestamp', and dropdown menus for 'Both NL & CR' and '115200 baud', along with a 'Clear output' button.

AP name



Signal Strength (dB)



1: Tenda_A32520 (-58)*

2: WXYZ_2.4GHz (-63)*

Authentication required



Example 2 : Hello Server

This example setups very simple web server on ESP32

Go to *File* → *Examples* → *WebServer* → *HelloServer*

```
const char* ssid = ".....";    //AP name  
const char* password = "....."; //Password
```

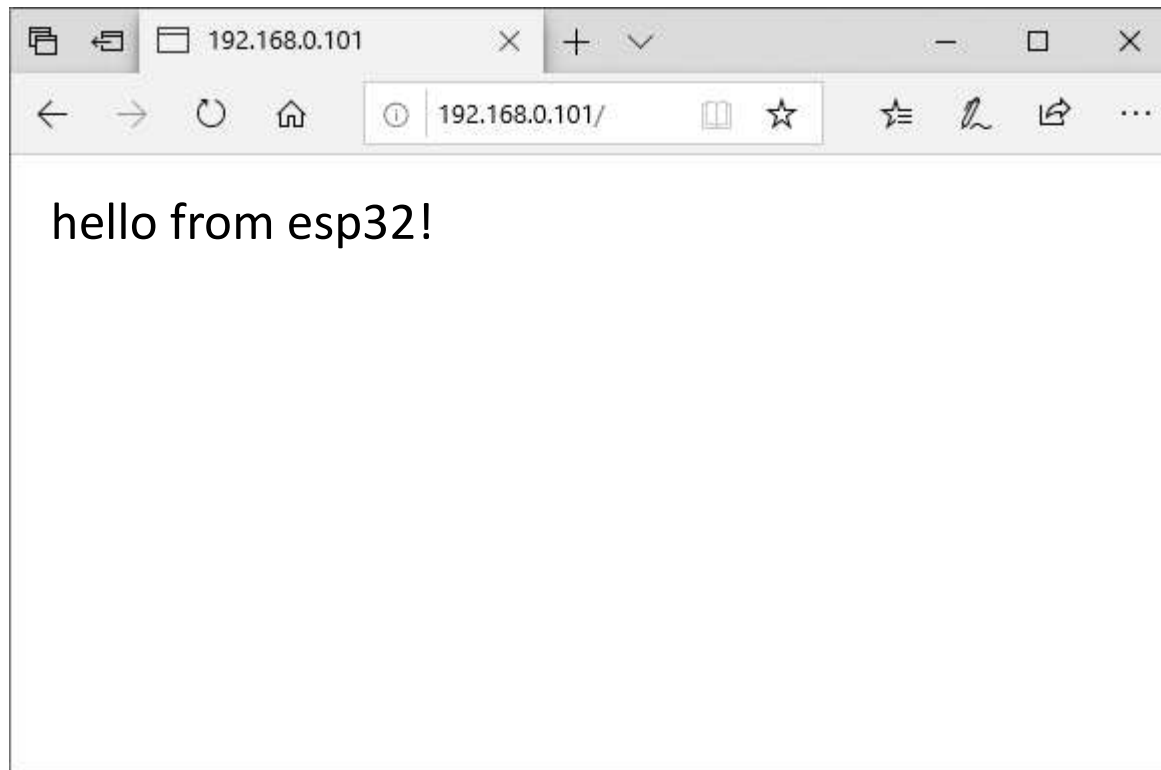


```
const char* ssid = "Note7";      //AP name  
const char* password = "qwerty1234"; //Password
```

Example 2 : Hello Server

Connected to Tenda_A32520
IP address: 192.168.0.101
MDNS responder started
HTTP server started

IP Address of ESP32

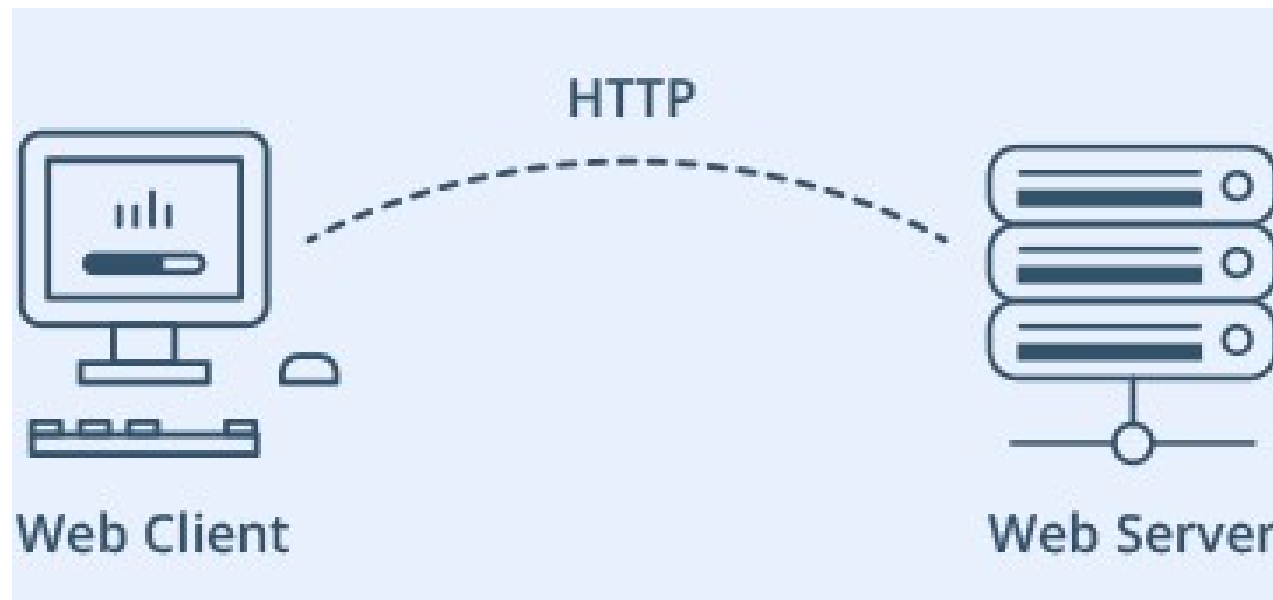


Open your browser and type above IP address in address bar

Example 2 : Hello Server

What is a Web server and how it works?

- Web server stores, processes and delivers web pages to Web clients.
- Web client is nothing but a web browser
- The communication between client and server takes place using a protocol called Hypertext Transfer Protocol (HTTP).



Browser (user)

ESP32

HTTP Protocol

Universal Resource Locator (URL)

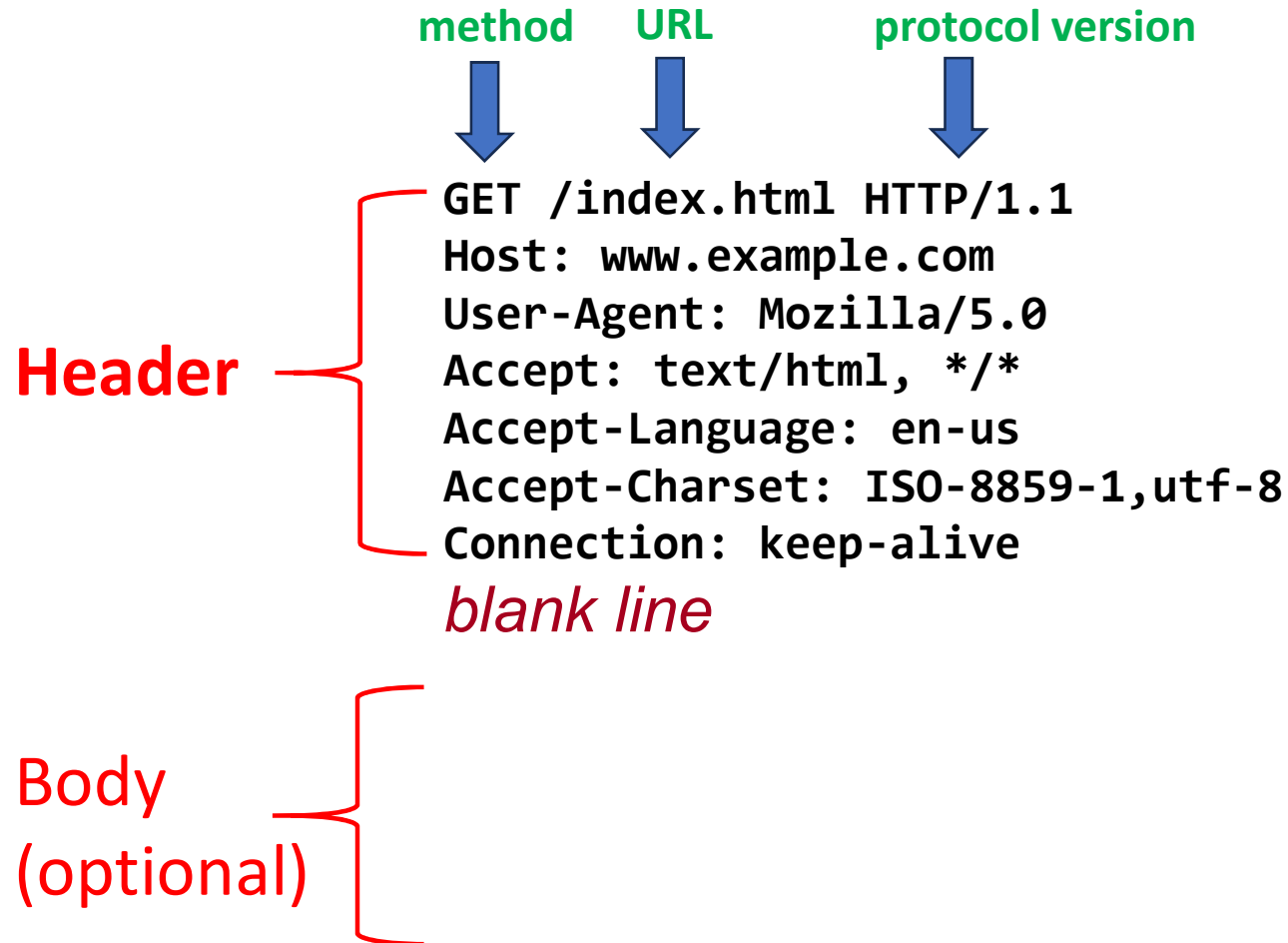


`http://www.example.com:80/index.html`

- To display page browser fetches the file `index.html` from a web server
Same as `www.example.com` (Defaults: port 80, file `index.html`, http protocol)
- HTTP (**H**yper**T**ext **T**ransport **P**rotocol)
- HTTP - Simple request-response **protocol** layered on TCP/IP
 1. Establish a **TCP/IP connection** to `www.example.com:80`
 2. Send a http GET **request** along connection
 3. Read from the connection the **response** from the web server

HTTP Protocol

Client (browser) sends HTTP request to server



HTTP Protocol

HTTP Methods (Verbs)

- GET - Fetch a URL
- HEAD - Fetch information about a URL
- PUT - Store to an URL
- POST - Send form data to a URL and get a response back
- DELETE - Delete a URL

GET and POST (forms) are commonly used.

REST APIs used GET, PUT, POST, and DELETE

Example 2 : Hello Server

HTTP Response from web server to client (browser)

	protocol version	status	status message
	↓	↓	↙
Header	HTTP/1.1 200 OK		
	Date: Thu, 24 Jul 2008 17:36:27 GMT		
	Server: Apache-Coyote/1.1		
	Content-Type: text/html; charset=UTF-8		
	Content-Length: 1846		
	<i>blank line</i>		
Body (html)	<?xml ... >		
	<!DOCTYPE html ... >		
	<html ... >		
	...</html>		

HTTP Protocol

Common HTTP Response Status Codes

200 OK Success

307 Temporary Redirect Redirection - Browser retries using Location header

404 Not Found Famous one

503 Service Unavailable Something crashed on the server

500 Internal Server Error Something is messed up on the server

501 Not Implemented Coming

400 Bad Request Use if web app sends bogus request

401 Unauthorized Use if user isn't logged in

403 Forbidden Use if even logging in wouldn't help

550 Permission denied Not allow to perform request

Example 3 : Advanced Web Server

This example setups more complicated web server on ESP32

Go to *File* → *Examples* → *WebServer* → *AdvancedWebServer*

```
const char* ssid = ".....";    //AP name  
const char* password = "....."; //Password
```

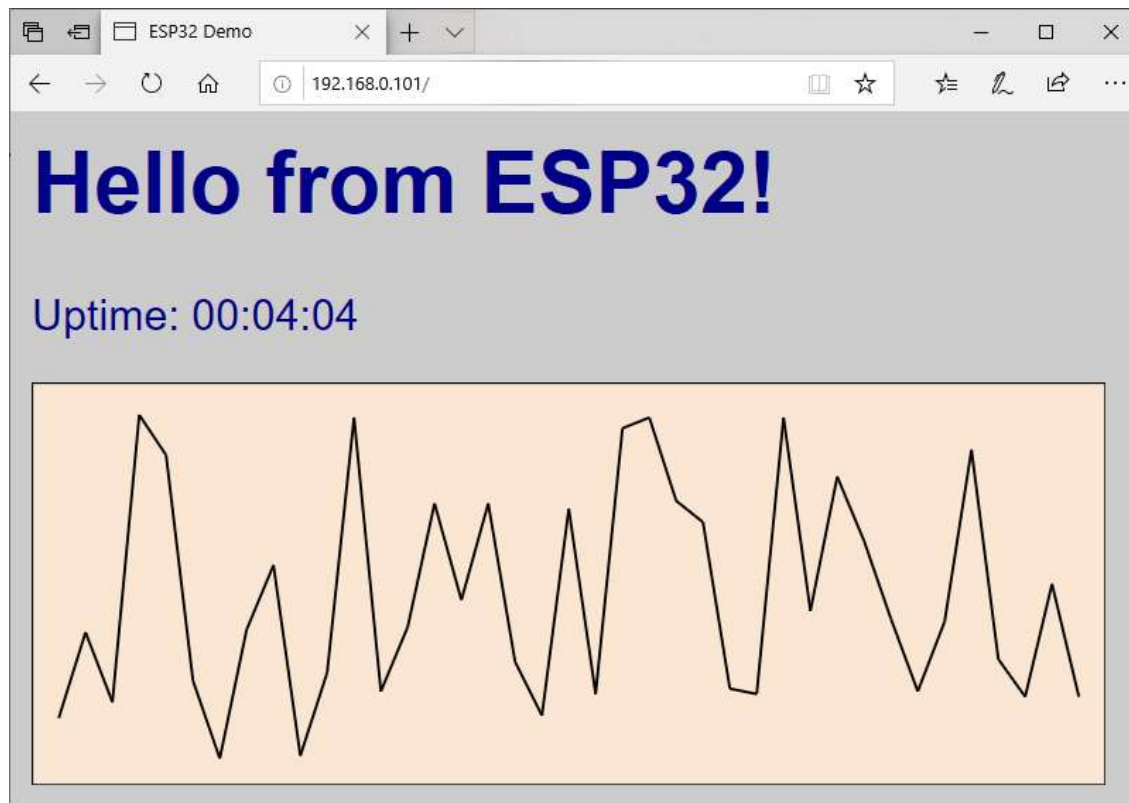


```
const char* ssid = "Note7";      //AP name  
const char* password = "qwerty1234"; //Password
```

Example 3 : Advanced Web Server

Connected to Tenda_A32520
IP address: 192.168.0.101
MDNS responder started
HTTP server started

IP Address of ESP32



Open your browser and type above IP address in address bar

Example 3 : Advanced Web Server

Concept

- Connect to AP and get IP address
- Setup web server
- Create web content
 - Text message & up timer
 - Random numbers and graphic plot
 - Timer
- Create connection handling
- Start web server & wait for connection

Example 4 : Web Server Controlled LED

This example shows how to control LED from web browser

Go to *File* → *Examples* → *WiFi* → *SimpleWiFiServer*

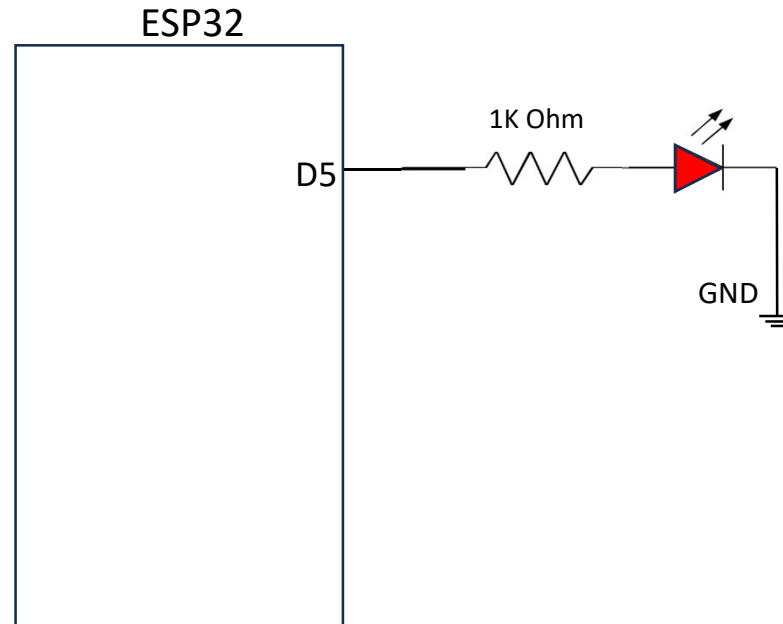
```
const char* ssid = ".....";    //AP name  
const char* password = "....."; //Password
```



```
const char* ssid = "Note7";      //AP name  
const char* password = "qwerty1234"; //Password
```

Example 4 : Simple Wi-Fi Server

Hardware setup



Example 4 : Simple Wi-Fi Server

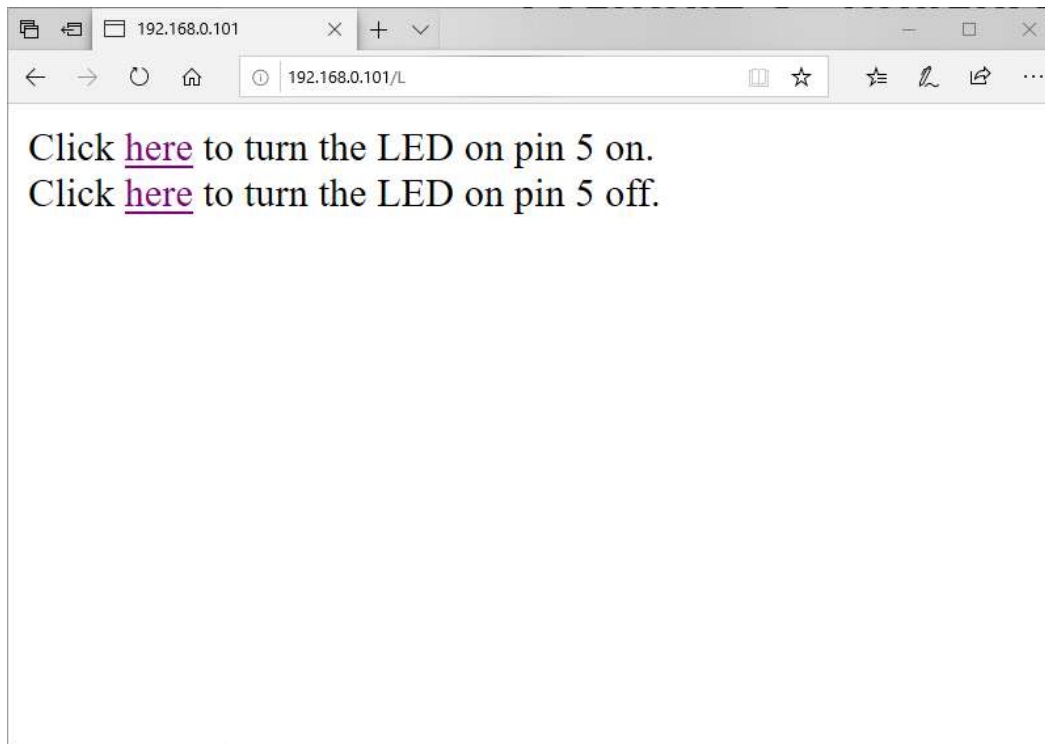
Connected to Tenda_A32520

IP address: 192.168.0.101

MDNS responder started

HTTP server started

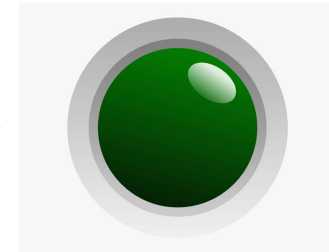
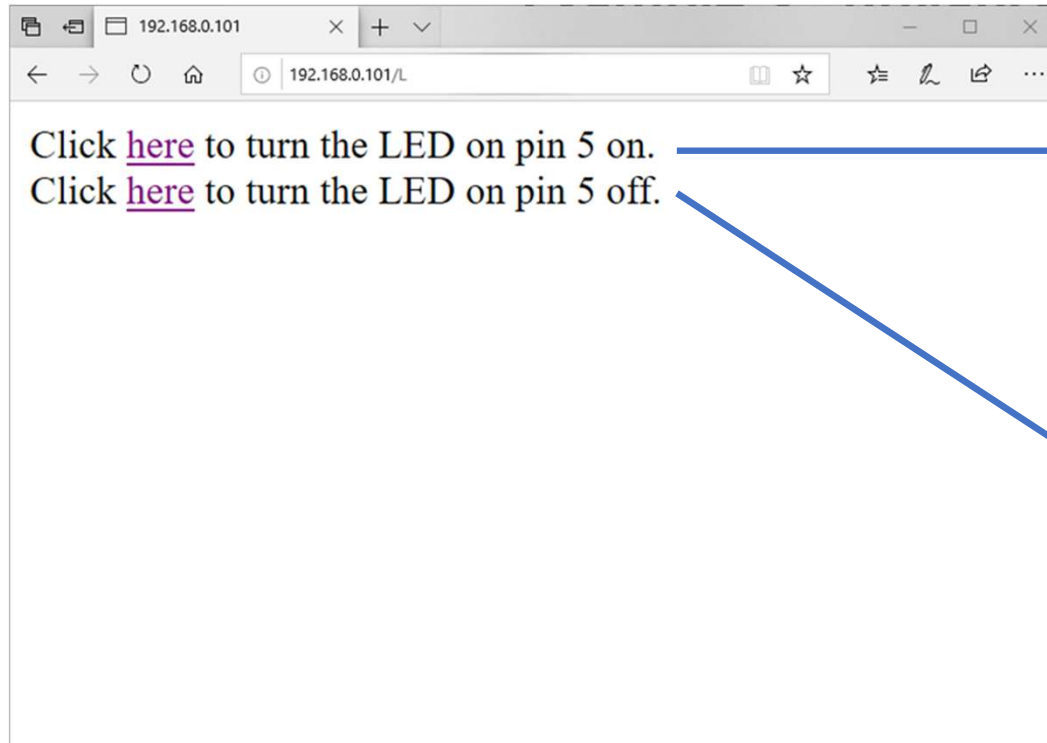
IP Address of ESP32



Open your
browser and
type above IP
address in
address bar

Example 4 : Simple Wi-Fi Server

User Interface



Example 4 : Simple Wi-Fi Server

Concept

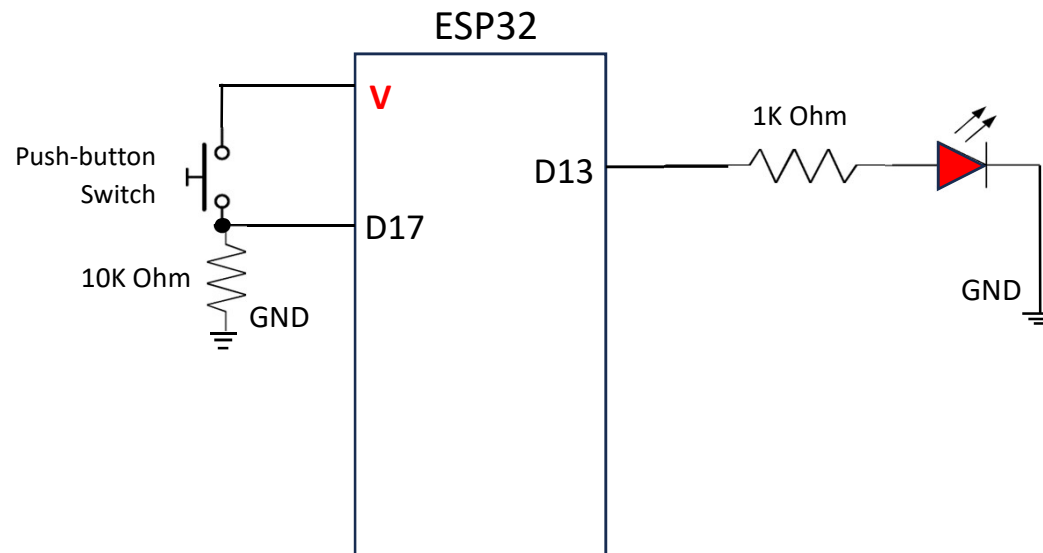
- Connect to AP and get IP address
- Setup web server
- Create web content
- Create connection handling
- Start web server & wait for connection
- If request is "GET /H" then Pin5 → "1"
- If request is "GET /L" then Pin5 → "0"

Example 5 : Switch Status Server

Hardware setup

This example shows how to use web server to display switch status

Open example file: [InputSW-WebServer.ino](#)



Example 5 : Switch Status Server

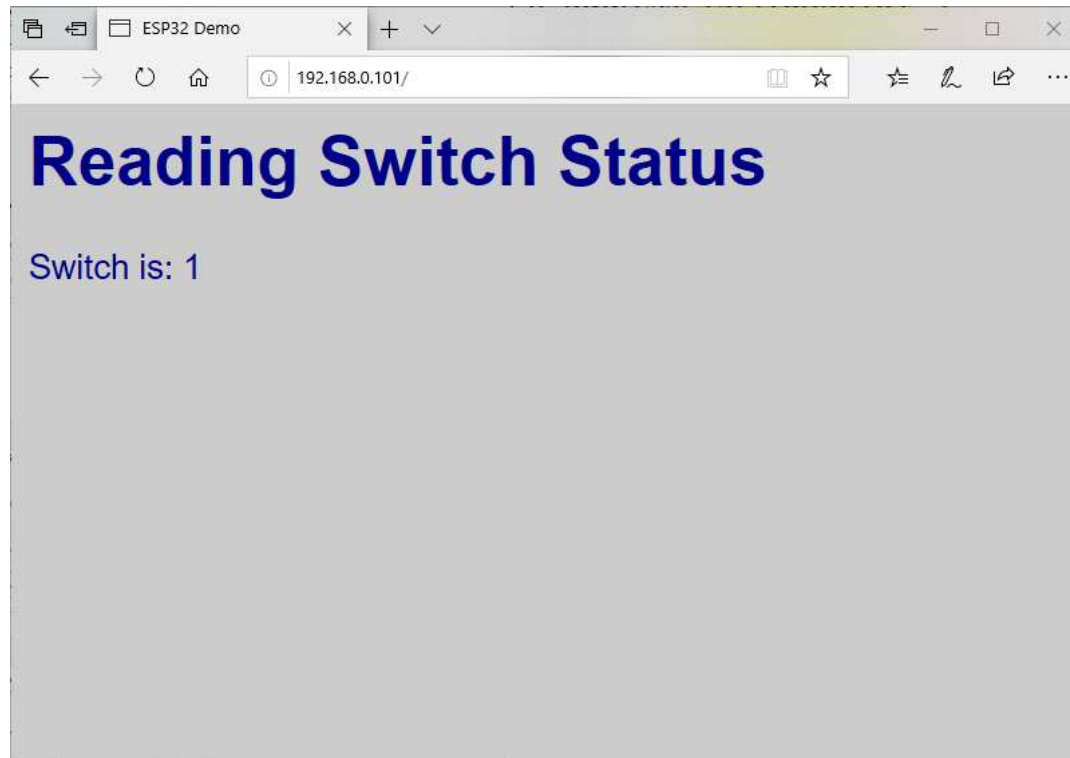
Connected to Tenda_A32520

IP address: 192.168.0.101

MDNS responder started

HTTP server started

IP Address of ESP32



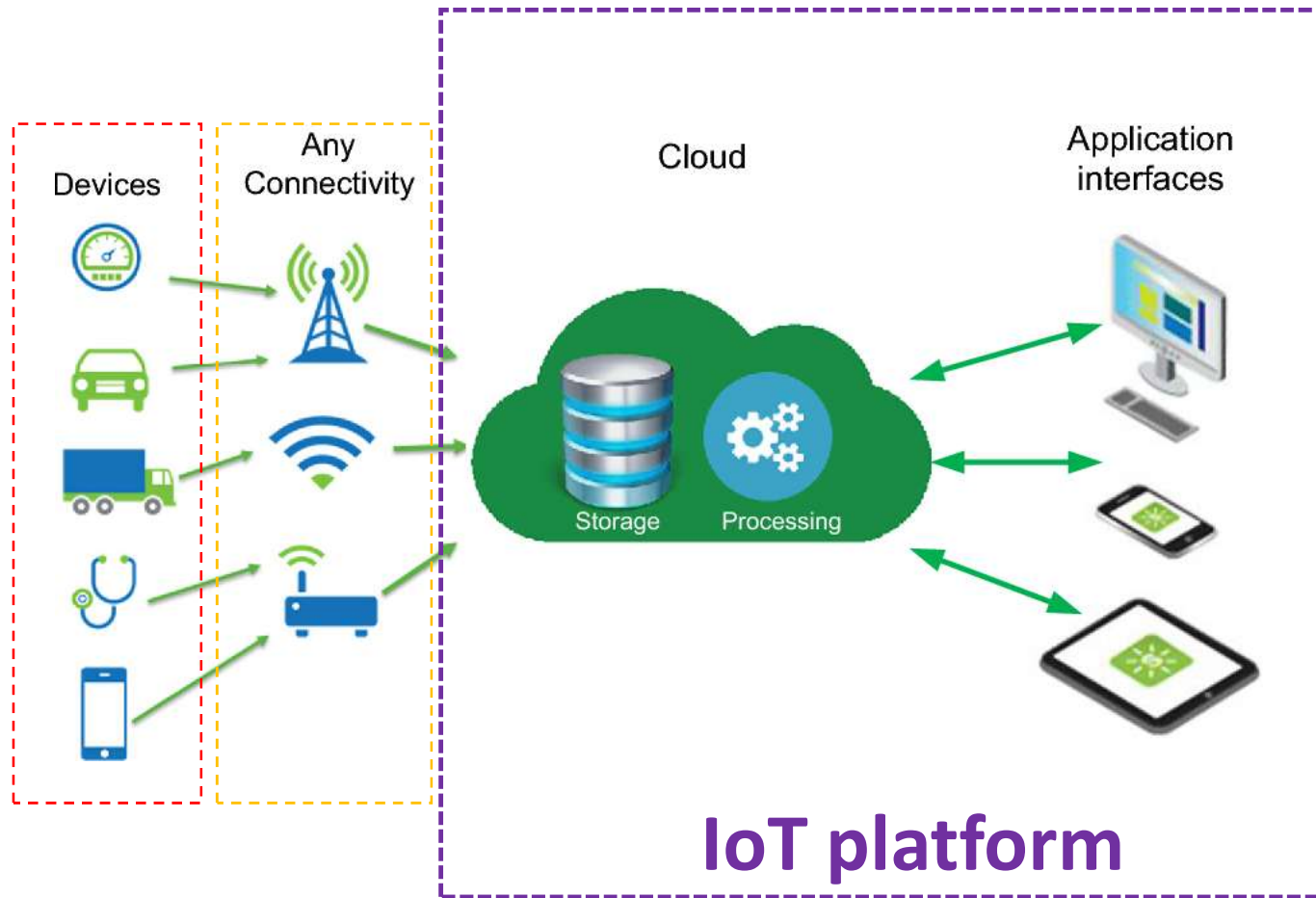
Open your
browser and
type above IP
address in
address bar

Example 5 : Switch Status Server

Concept

- Connect to AP and get IP address
- Setup web server
- Create web content
- Create connection handling
- Start web server & wait for connection
- Keep scanning the input pin and update web content

Architecture of IoT

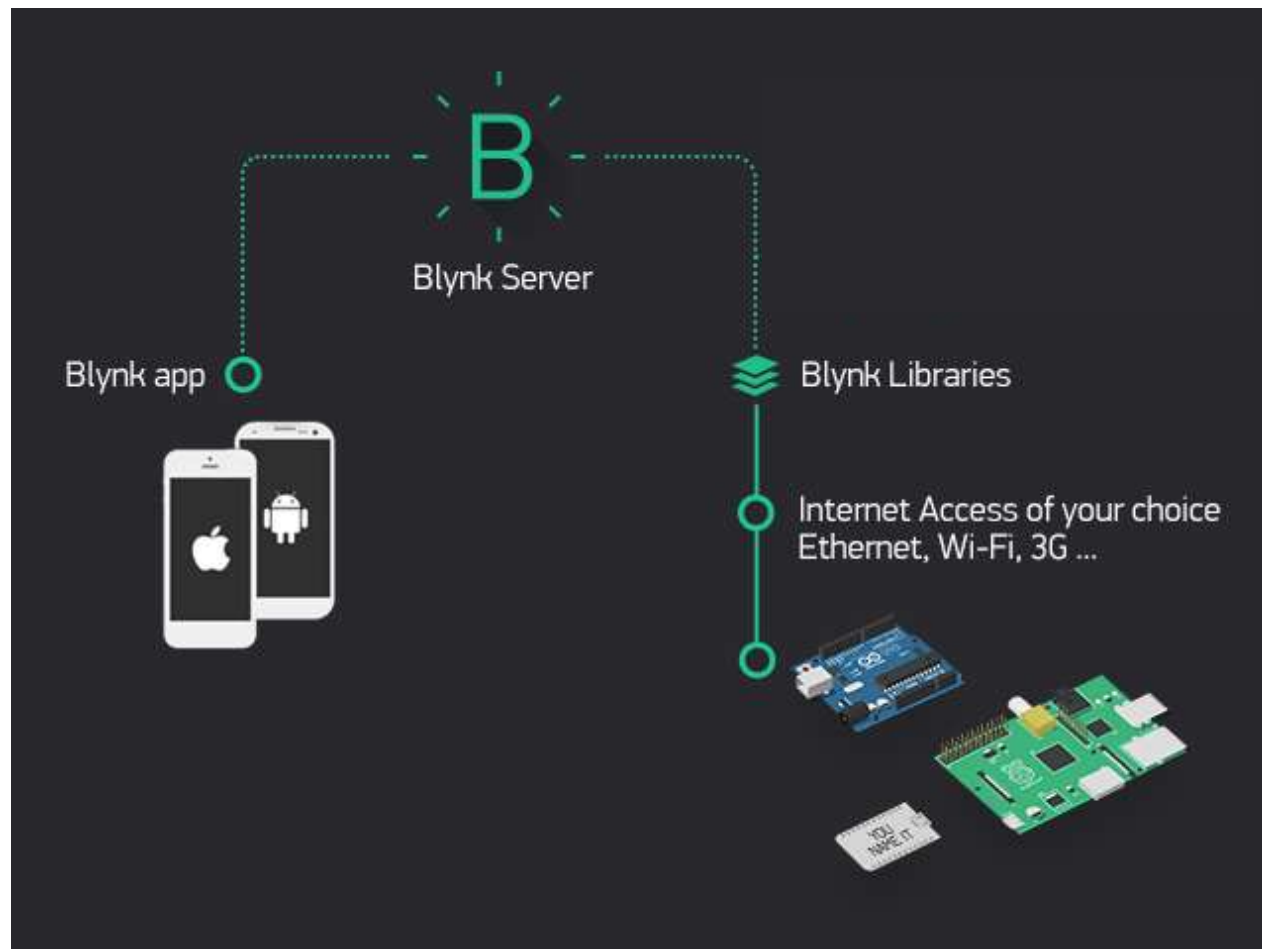


Basic IoT application development

- IoT applications require a lot of customized HW and SW
- Common parts in IoT applications are user interface
- There are available platform for creating simple IoT apps
 - Blynk
 - NetPie (by NECTEC)
 - ThingWorx

IoT Applications Development using Blynk

- Blynk is an IoT Application development platform consisting of:
- User interface for desktop and mobile devices (Android/iPhone)
- Connection library for various IoT hardware platforms



Blynk Applications Development

- Signup for Blynk by going to: <http://blynk.io>
- Type your email address to proceed to the next step



Sign Up

Welcome! Fill in your email address and we will send an account activation link.

EMAIL



☒ I agree to [Terms and Conditions](#) and accept [Privacy Policy](#)

Sign Up

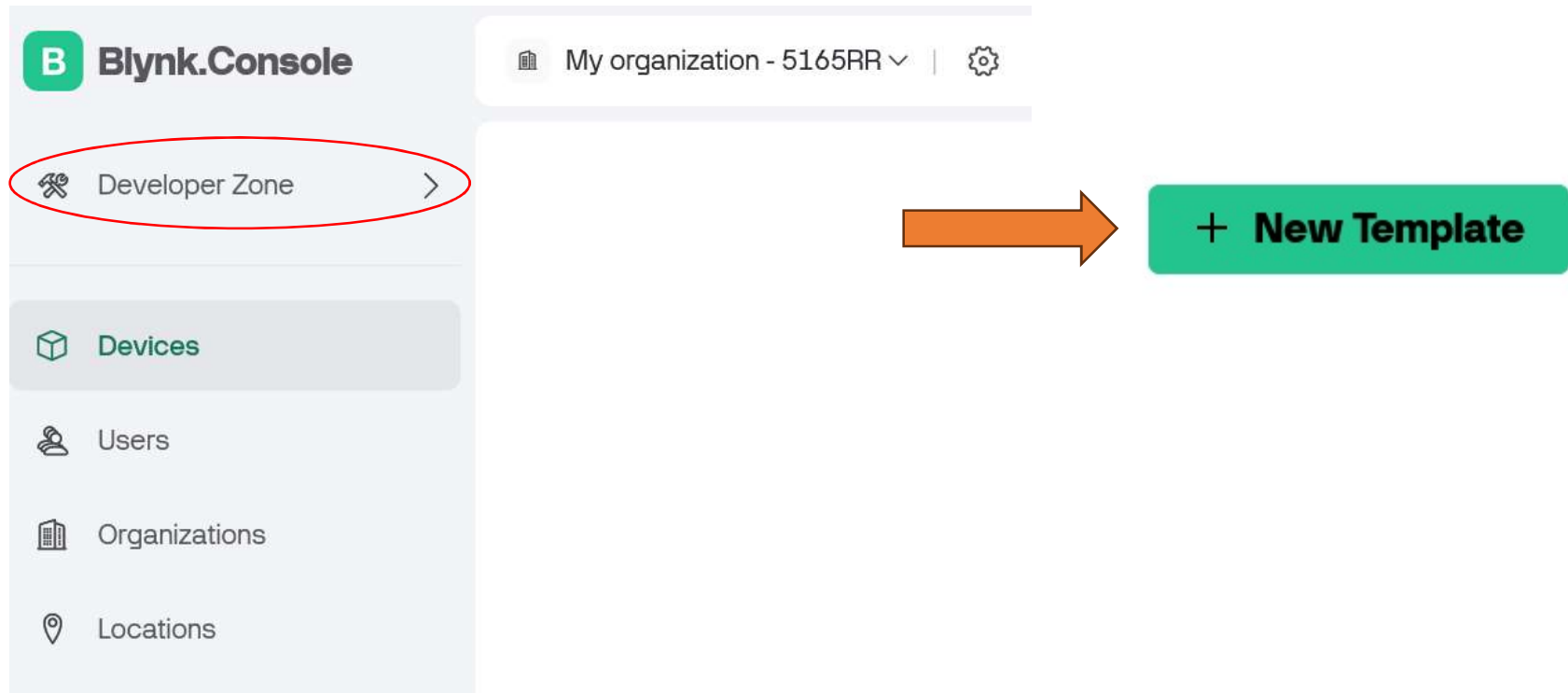
[Back to Login](#)

Blynk Applications Development

- Open your email, then set a new password
- Go back to main screen, log in using your new password
- Once you log in, you will see Blynk Console
- Steps to create a Blynk application are as follows:
 - 1) Create new template in Blynk console
 - 2) Create device for your template
 - 3) Create user interface in Blynk mobile app
 - 4) Build your circuit and write code for ESP32

Creating a new template in Blynk Console

- Click on Developer Zone on the left menu to go to template
- Then click on New template



Creating a new template in Blynk Console

- Under Name, type “LED Control” then select ESP32 and WiFi
- Click on Done button

Create New Template

NAME

LED Control 11 / 50

HARDWARE CONNECTION TYPE

ESP32 WiFi

DESCRIPTION

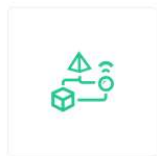
Description

0 / 128

Cancel Done

Creating a new template in Blynk Console

- The next step is to create new datastream by clicking on **Datastreams** tab and then **New datastream** button



LED Control

Home

Datastreams

Web Dashboard

Metadata

Connection Lifecycle

Events & Notifications

Mobile Dashboard



Datastreams

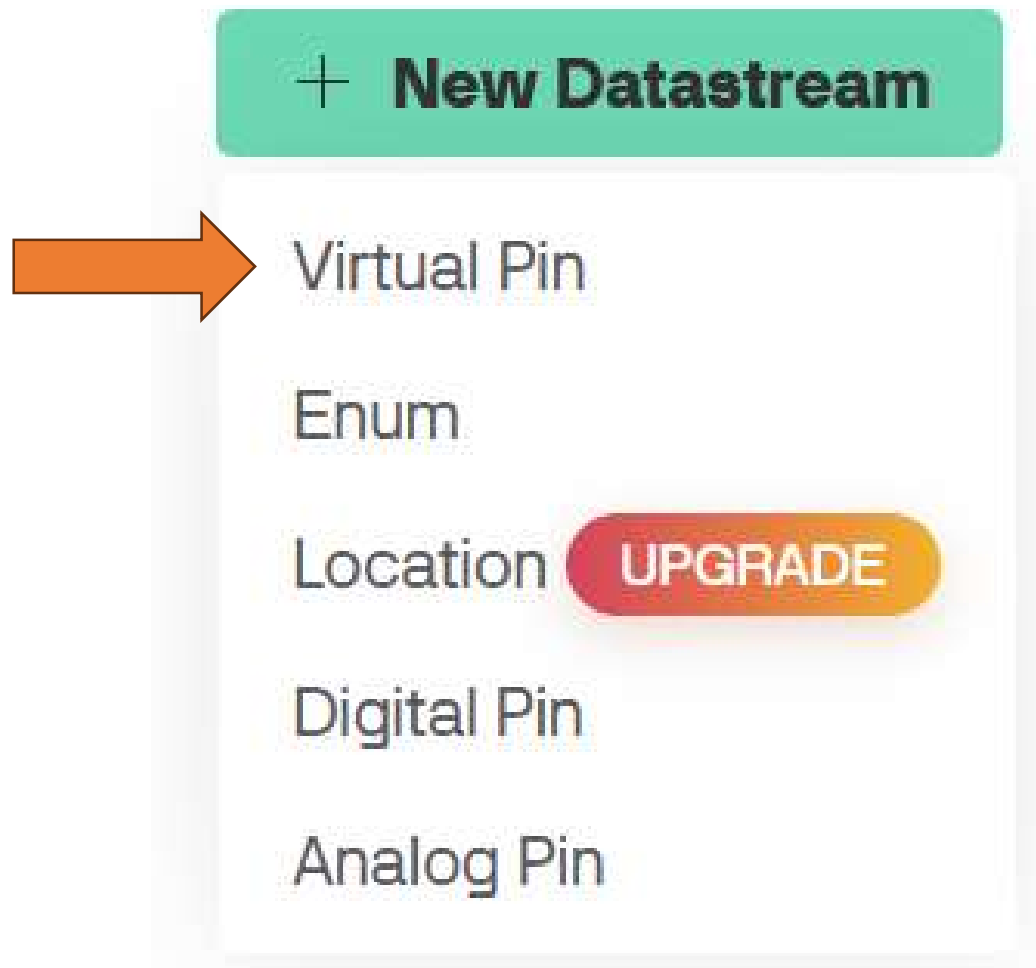
Datastreams is a way to structure data that regularly flows in and out from device. Use it for sensor data, any telemetry, or actuators.



+ New Datastream

Creating a new template in Blynk Console

- After new datastream is created, choose Virtual Pin



Creating a new template in Blynk Console

- At the bottom of Virtual Pin Datastream setup screen, click on Create button

Virtual Pin Datastream

General

Expose to Automations

NAME	ALIAS	
 Integer V0	Integer V0 	
PIN	DATA TYPE	
V0	Integer	
UNITS		
None		
MIN	MAX	DEFAULT VALUE
0	1	0

☐ Enable history data

☐ ADVANCED SETTINGS

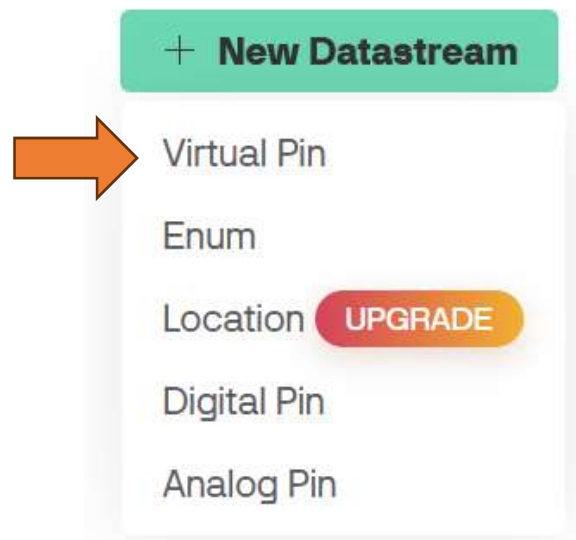
Cancel

Create



Creating a new template in Blynk Console

- Repeat creating Virtual Pin Datastream again by clicking at New datastream



Virtual Pin Datastream

General Expose to Automations

NAME		ALIAS	
	Integer V1	Integer V1	
PIN		DATA TYPE	
V1		Integer	
UNITS			
None			
MIN	MAX	DEFAULT VALUE	
0	1	0	

☐ Enable history data

ADVANCED SETTINGS

Cancel

Save

Creating a new template in Blynk Console

- Now Datastreams tab should look like this:

Home **Datastreams** Web Dashboard Metadata Connection Lifecycle Events & Notifications Mobile Dashboard

Search datastream + New Datastream

2 Datastreams

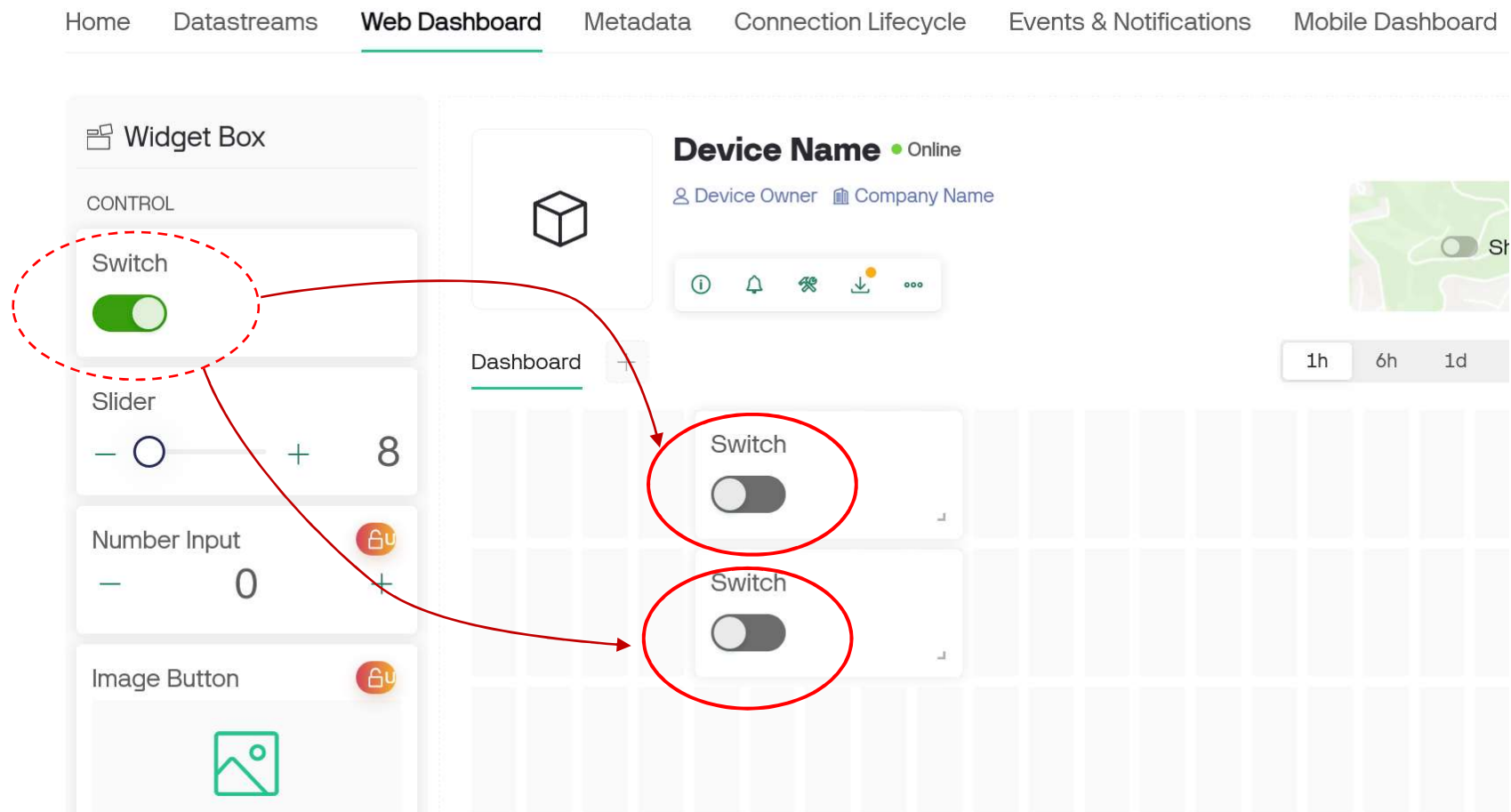
☐	Id	Name	Alias	Color	Pin	Data Type	Units	Is Raw	Min	Max	Actions
	1	Integer V0	Integer V0		V0	Integer		false	0	1	
	2	Integer V1	Integer V1		V1	Integer		false	0	1	

2 Datastreams

☐	Id	Name	Alias	Color	Pin	Data Type
	1	Integer V0	Integer V0		V0	Integer
	2	Integer V1	Integer V1		V1	Integer

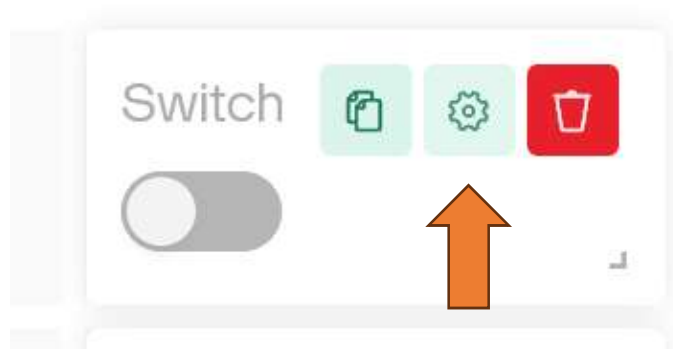
Creating a new template in Blynk Console

- Go to Web Dashboard then drag Switch from widget box and place on the Dashboard



Creating a new template in Blynk Console

- Move mouse pointer over Switch widget, then click on setting icon



Creating a new template in Blynk Console

- Under Title, type LED1, then set Datastream to Integer V0 [V0] then click on Save

Switch Settings

TITLE (OPTIONAL)



Datastream

ON VALUE

OFF VALUE

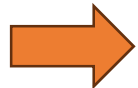


Creating a new template in Blynk Console

- Repeat this process for setting the other Switch

Switch Settings

TITLE (OPTIONAL)



LED2

DataStream



Integer V1 (V1)



ON VALUE

1

OFF VALUE

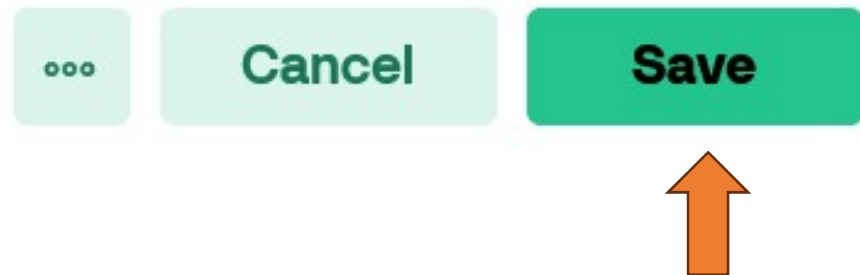
0



- Click on Save when finish

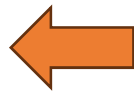
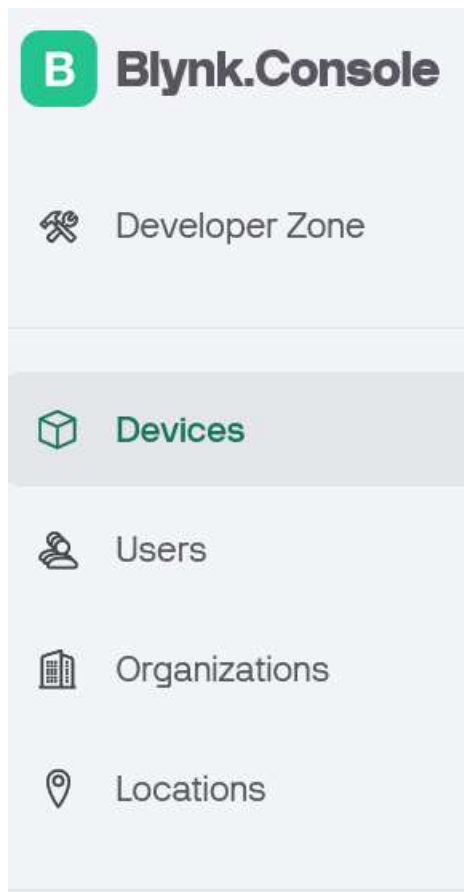
Creating a new template in Blynk Console

- Finally, click on Save button on the right



Creating device in Blynk Console

- Next, choose Devices from menu on the left
- Then, click on + New Device

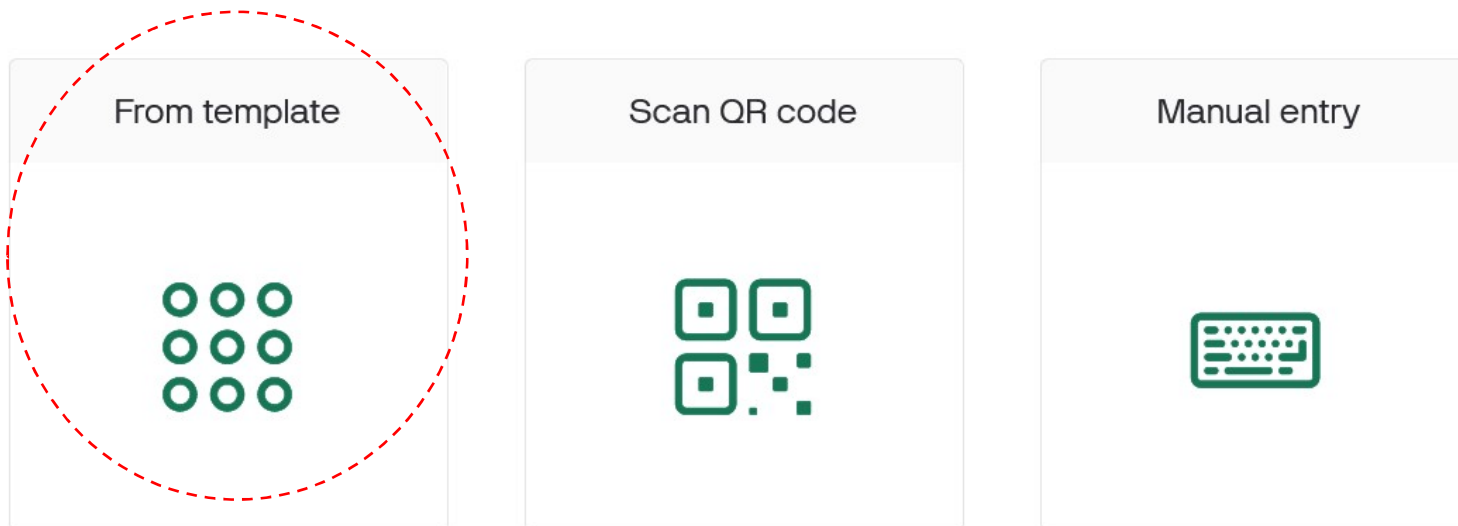


Creating device in Blynk Console

- Choose From template

New Device

Choose a way to create new device



Point on the cards to see instructions

Creating device in Blynk Console

- Choose the template we have created
- Choose **LED Control**, then click on Create

New Device

Create new device by filling in the form below

TEMPLATE

LED Control



DEVICE NAME

LED Control

11 / 50

Cancel

Create

Creating device in Blynk Console

- **New Device Created!** message will be displayed
- We need to copy the Template ID, Name and AuthToken into our sketch

New Device Created! X

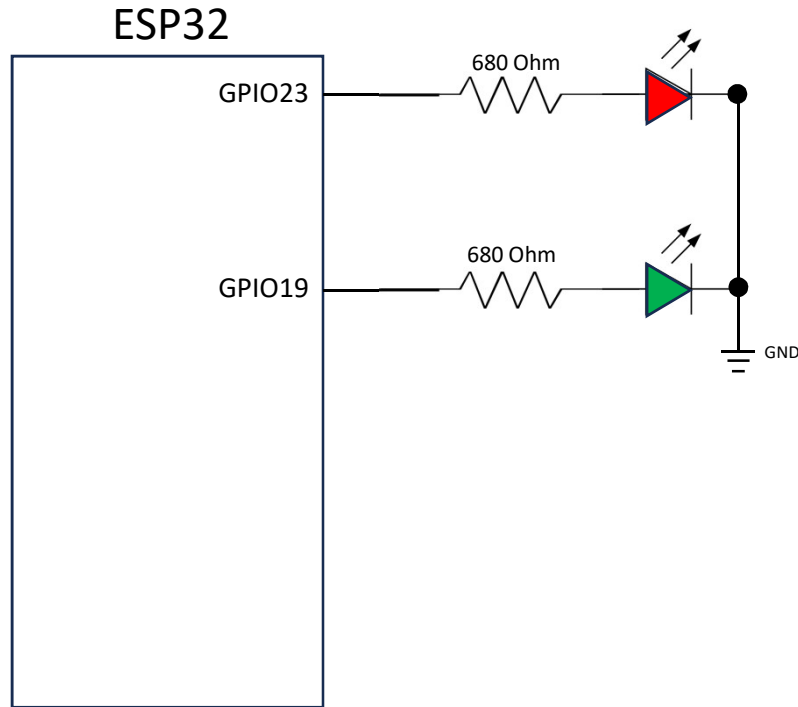
```
#define BLYNK_TEMPLATE_ID "TMPL6M-dQi4Bk"  
#define BLYNK_TEMPLATE_NAME "LED Control"  
#define BLYNK_AUTH_TOKEN  
"BrkURlRodfuvGufReSllV7YxSUiRwJbN"
```

Template ID, Template Name, and AuthToken should be declared at the very top of the firmware code.

[Documentation](#)

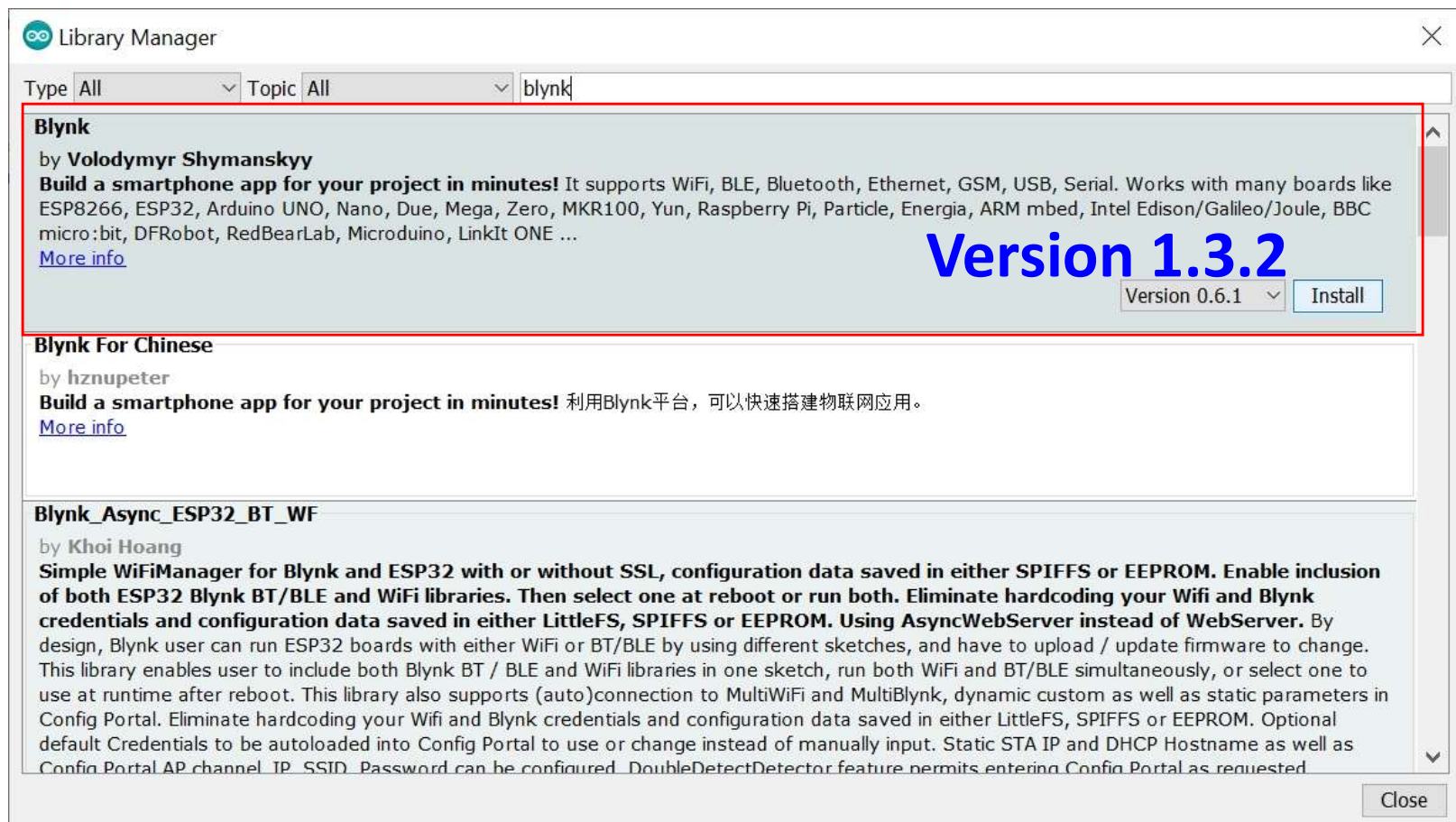
[Copy to clipboard](#)

Test circuit for Blynk Application



Install Blynk Library in Arduino IDE

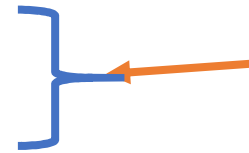
- Go to menu Tools → Manage Libraries...
- Type **Blynk** in search box
- Click install on Blynk library



Blynk sketch for ESP32

- Download Blynk_LED_ESP32.ino sketch from <https://github.com/watisleelapatra/ESP32-Advance>
- Open the sketch in Arduino IDE
- Paste the following lines in the sketch

```
#define BLYNK_TEMPLATE_ID " "  
#define BLYNK_TEMPLATE_NAME " "  
#define BLYNK_AUTH_TOKEN " "
```



Information from
Blynk Console

Example

```
#define BLYNK_TEMPLATE_ID "TMPL6M-dQi4Bk"  
#define BLYNK_TEMPLATE_NAME "LED Control"  
#define BLYNK_AUTH_TOKEN  
"BrkURlRodfuvGufReSllV7YxSUiRwJbN"
```

Blynk sketch for ESP32

- Add your WiFi AP and password in the sketch

```
char ssid[] = " ";  
char pass[] = " ";
```

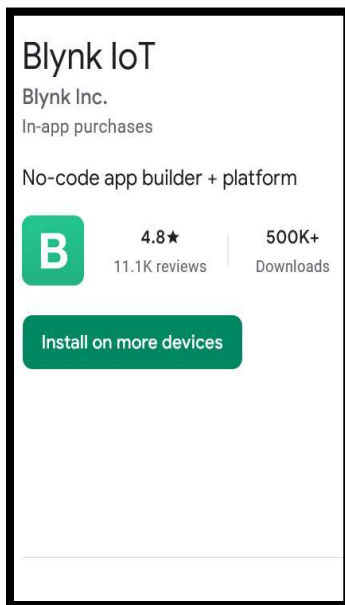
← Wi-Fi AP name

← Password for Wi-Fi AP

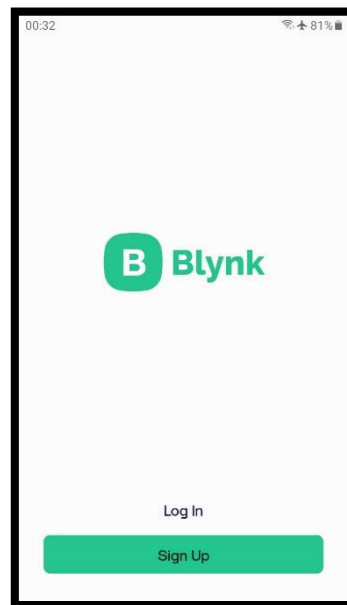
- Then download the sketch

Install Blynk IoT Mobile Application

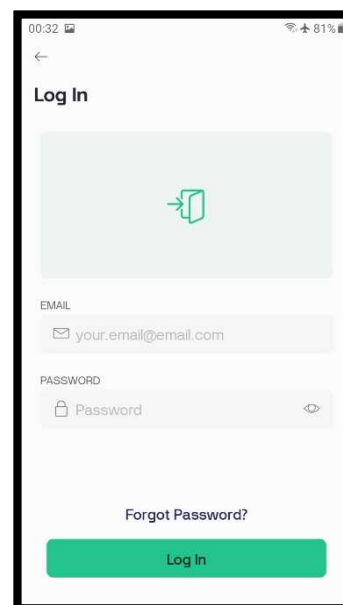
- Go to App Store (iPhone) or Google Play (Android)
- Search and install **Blynk IoT** on your mobile phone
- After installation, Login using email you for signing up
- There will be the created Template, Choose LED Control



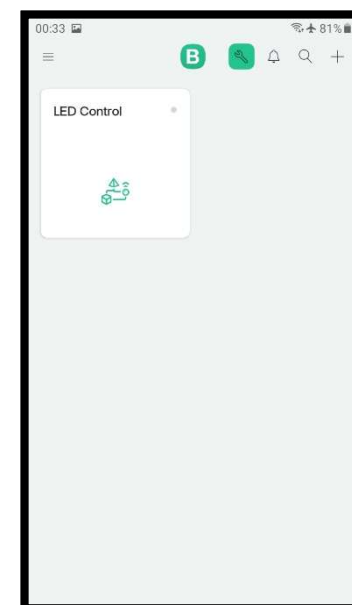
1



2



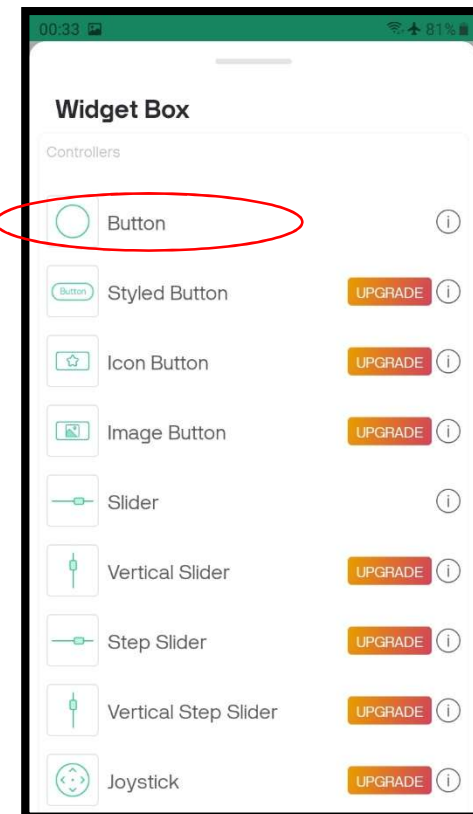
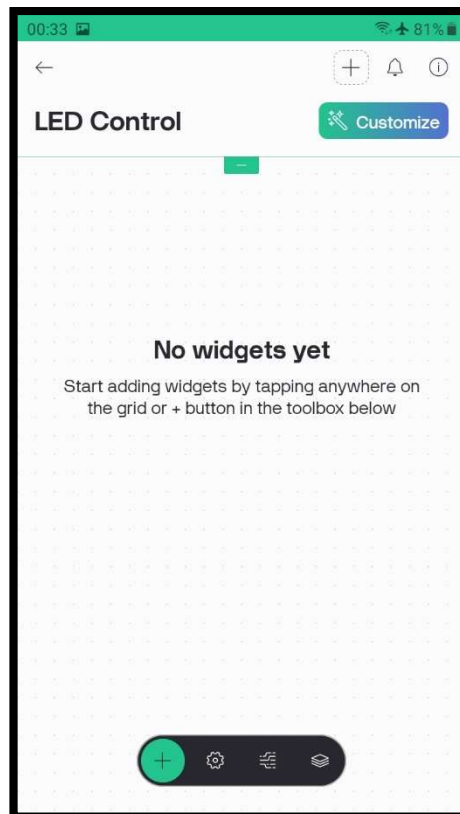
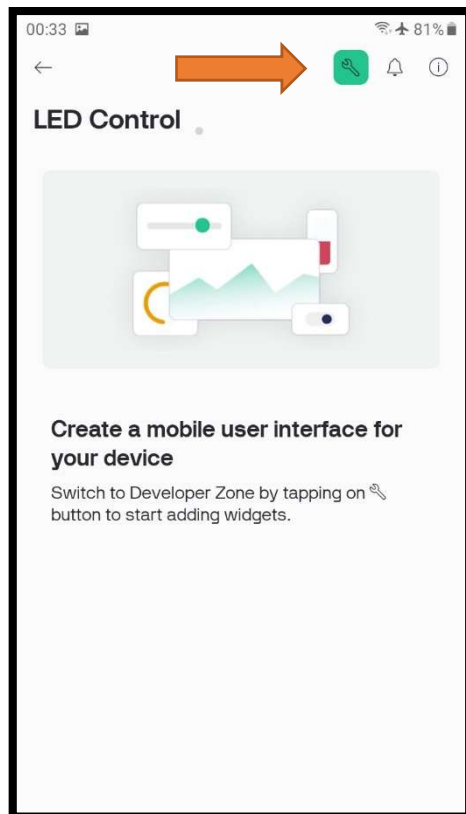
3



4

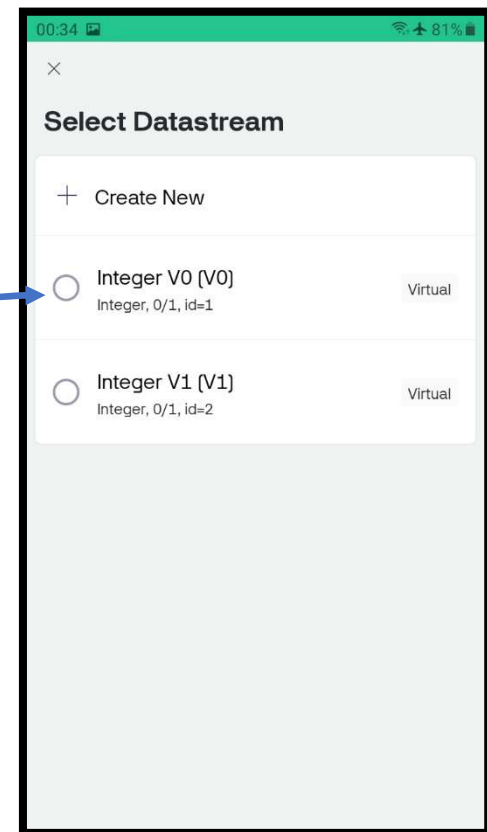
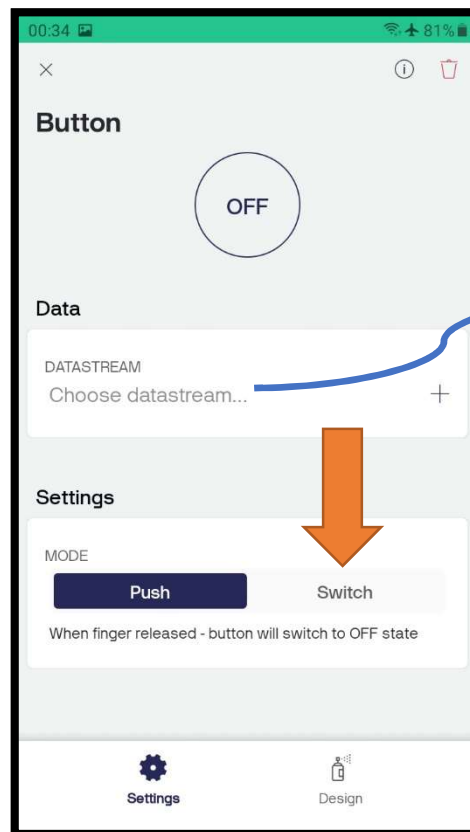
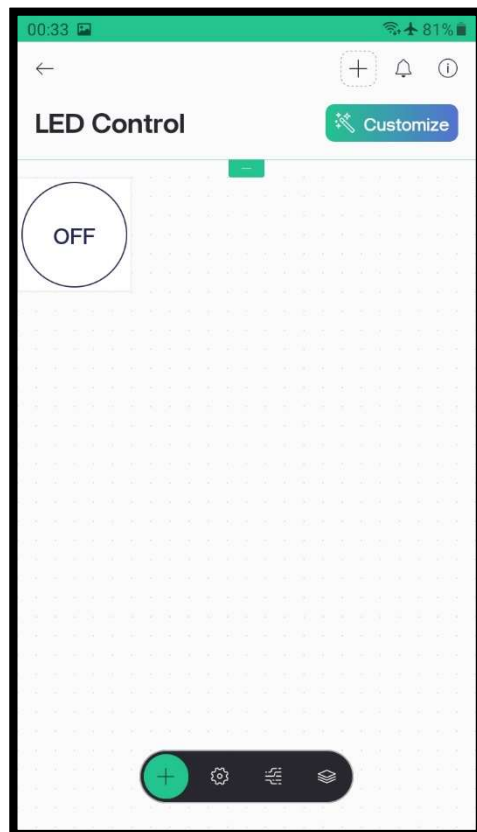
Creating Mobile Application

- After choosing LED Control template, click on the wrench icon
- Add widget by tapping  icon
- Then choose Button



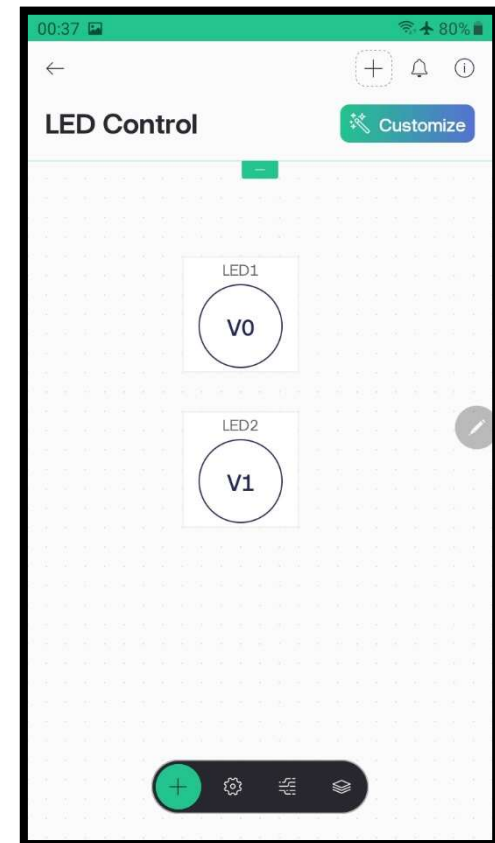
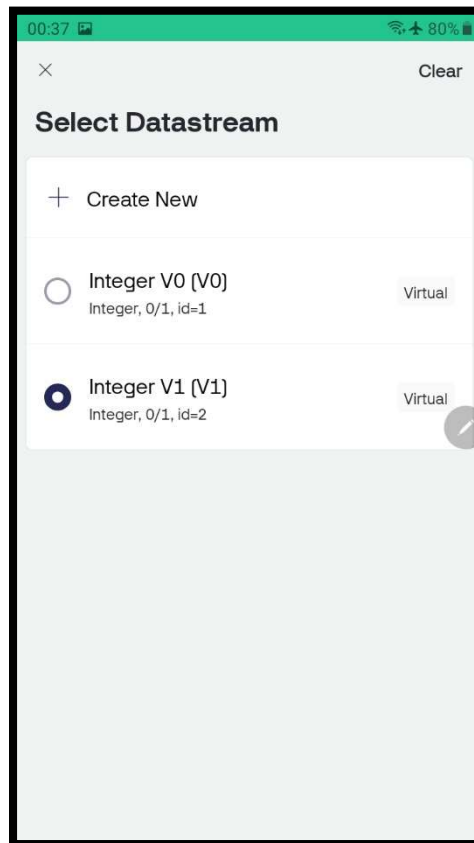
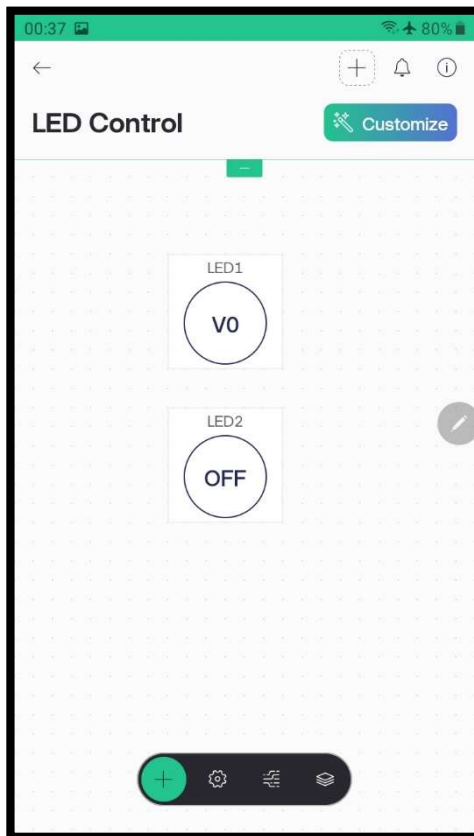
Creating Mobile Application

- A button will be shown, then tap on the button
- Change MODE from **Push** to **Switch**
- Then set DATASTREAM to **Integer V0 [V0]**



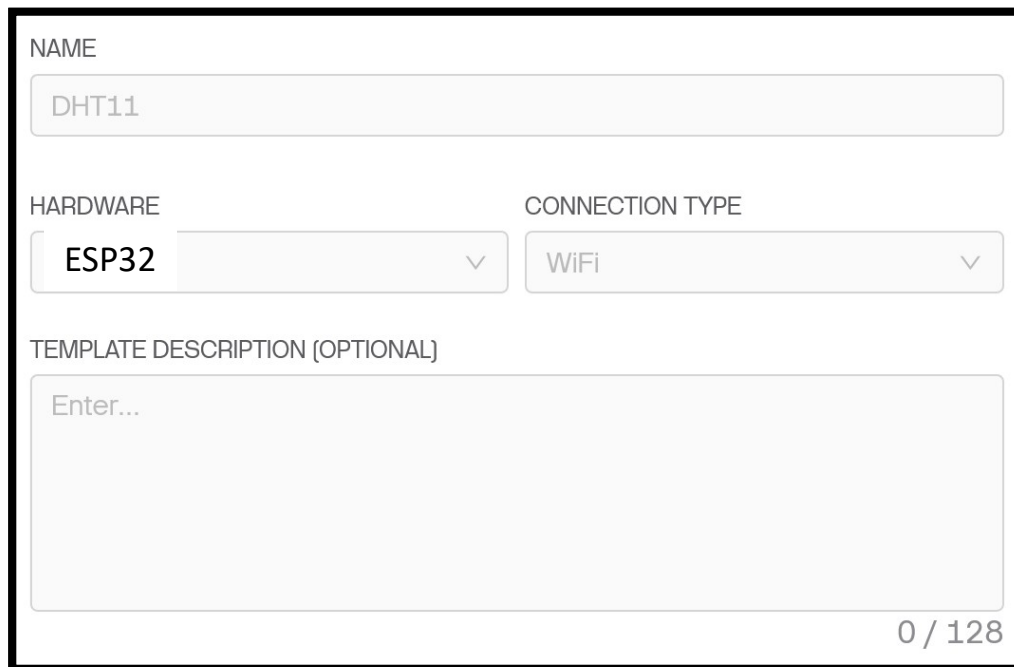
Creating Mobile Application

- Repeat the process, tap on  then choose Button
- Change Mode to Switch and set DATASTREAM to **Integer V1 [V1]**
- Tap  icon (top left of the screen) to go back to Application main screen



Sending DHT11 data to Blynk IoT App

- Create a new template in Blynk Console, go to Developer zone and click on
New template
- Type DHT11 under Name, choose ESP32 for Hardware and WiFi for
Connection Type



A screenshot of the Blynk Console 'New template' form. The form is enclosed in a black border and contains the following fields:

- NAME:** A text input field containing 'DHT11'.
- HARDWARE:** A dropdown menu with 'ESP32' selected.
- CONNECTION TYPE:** A dropdown menu with 'WiFi' selected.
- TEMPLATE DESCRIPTION (OPTIONAL):** A large text area with the placeholder text 'Enter...'. At the bottom right of this area, the character count '0 / 128' is displayed.

Sending DHT11 data to Blynk IoT App

- Add Virtual Pin for DATASTREAM and set other parameters as follow:

The screenshot shows the Blynk IoT App configuration interface for a Virtual Pin. The 'General' tab is selected. The configuration includes:

- NAME:** Integer V0
- ALIAS:** Integer V0
- PIN:** V0
- DATA TYPE:** Integer
- UNITS:** Celsius, °C
- MIN:** 0
- MAX:** 80
- DEFAULT VALUE:** Default Value

Red circles highlight the DATA TYPE, UNITS, and MAX fields.

Sending DHT11 data to Blynk IoT App

- Add 2nd DATASTREAM by choosing Virtual Pin and set parameters as follow:

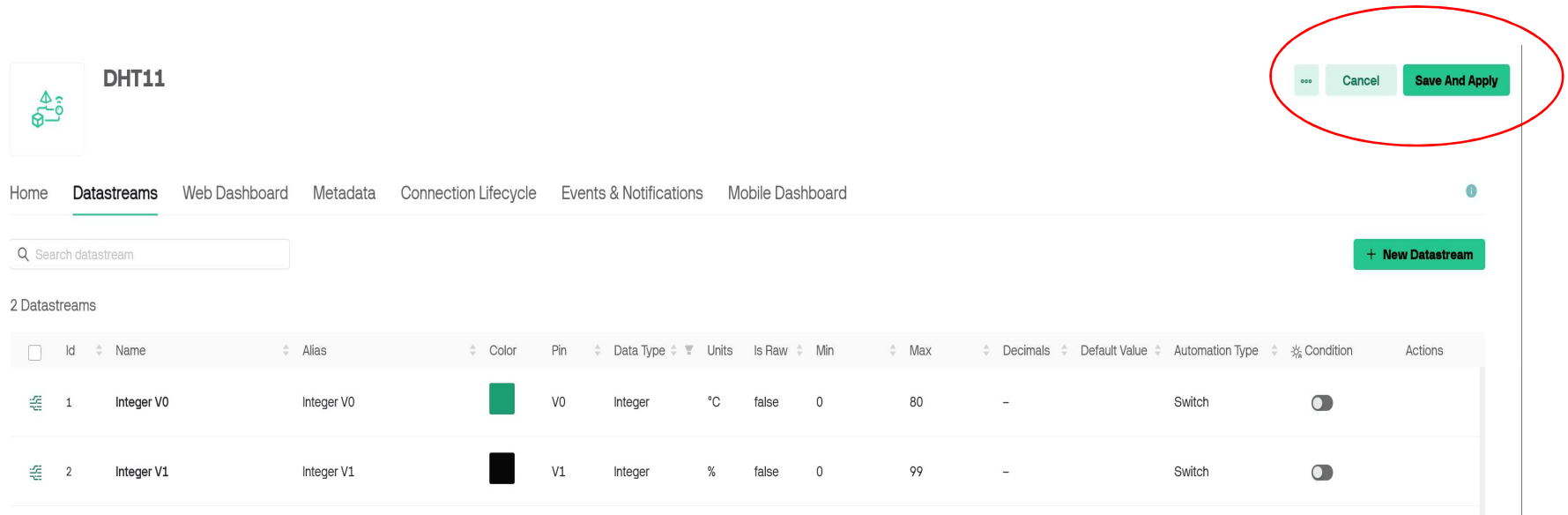
The screenshot shows the 'General' configuration tab for a data stream in the Blynk app. The interface includes the following fields and settings:

- NAME:** Integer V1
- ALIAS:** Integer V1
- PIN:** V1
- DATA TYPE:** Integer
- UNITS:** Percentage, %
- MIN:** 0
- MAX:** 99
- DEFAULT VALUE:** Default Value

Red circles highlight the 'DATA TYPE', 'UNITS', and 'MAX' fields, indicating the required configuration for the data stream.

Sending DHT11 data to Blynk IoT App

- Finally, click on **Save And Apply**



The screenshot shows the Blynk IoT App interface for configuring a DHT11 sensor. The top navigation bar includes links to Home, Datastreams, Web Dashboard, Metadata, Connection Lifecycle, Events & Notifications, and Mobile Dashboard. The 'Datastreams' tab is active. Below the navigation bar, there is a search bar labeled 'Search datastream' and a '+ New Datastream' button. The main content area displays a table of 2 Datastreams. The first datastream is 'Integer V0' with a green color, and the second is 'Integer V1' with a black color. Both datastreams are configured as Integer type, with units of °C and %, respectively. The 'Automation Type' is set to 'Switch' for both, and the 'Condition' is set to 'On' (indicated by a toggle switch). The 'Save And Apply' button is highlighted with a red circle.

DHT11

Home **Datastreams** Web Dashboard Metadata Connection Lifecycle Events & Notifications Mobile Dashboard

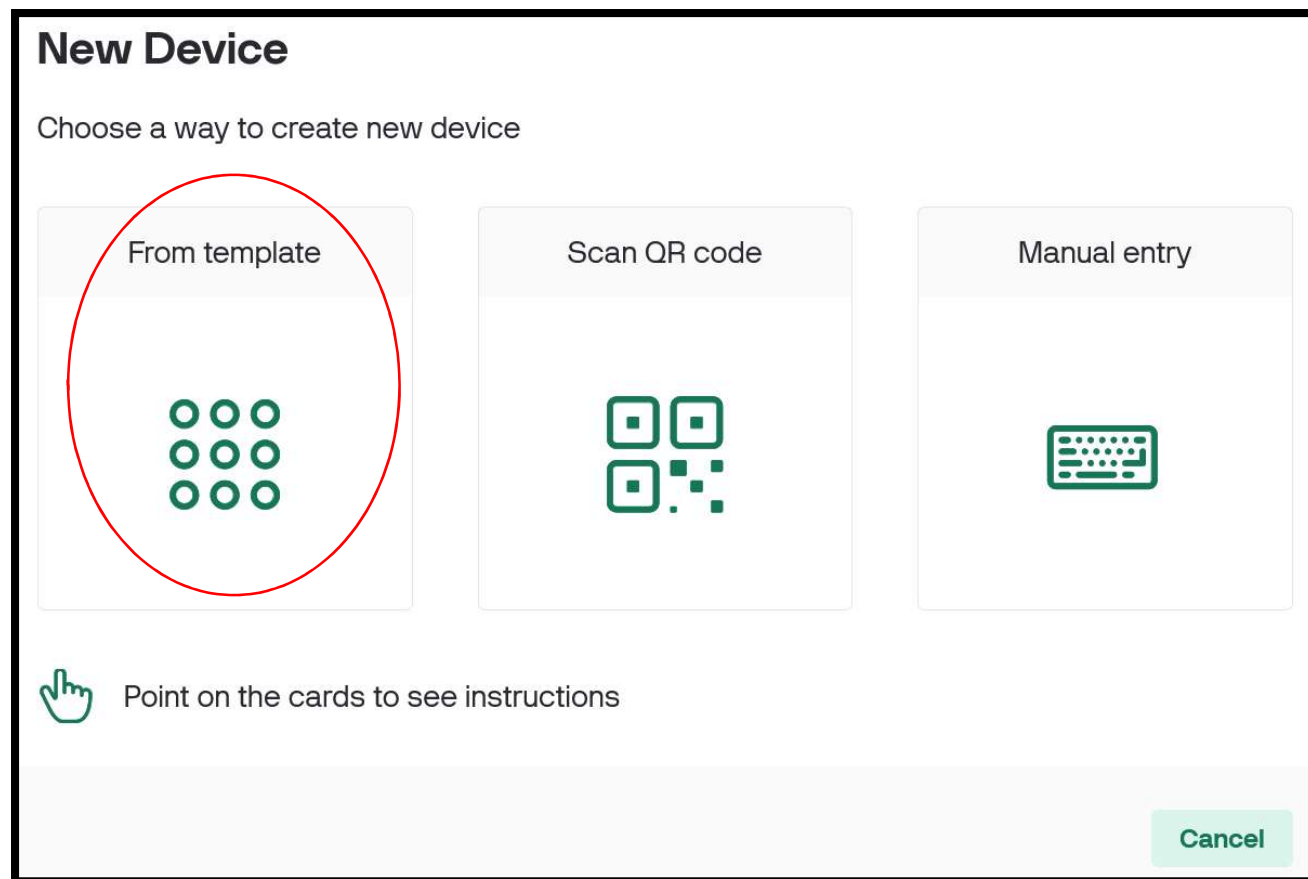
Search datastream + New Datastream

2 Datastreams

	Id	Name	Alias	Color	Pin	Data Type	Units	Is Raw	Min	Max	Decimals	Default Value	Automation Type	Condition	Actions
	1	Integer V0	Integer V0		V0	Integer	°C	false	0	80	-		Switch		
	2	Integer V1	Integer V1		V1	Integer	%	false	0	99	-		Switch		

Sending DHT11 data to Blynk IoT App

- Next, go to Devices menu and click on **+New Device**
- Click on **From template**



Sending DHT11 data to Blynk IoT App

- Choose DHT11 for Template , then click **Create**

New Device

Create new device by filling in the form below

TEMPLATE

DHT11

DEVICE NAME

DHT11 5 / 50

Cancel

Create

Sending DHT11 data to Blynk IoT App

- Template ID, Template Name and Token will be generated
- Click on Copy to clipboard, then paste these information in your sketch

New Device Created!

×

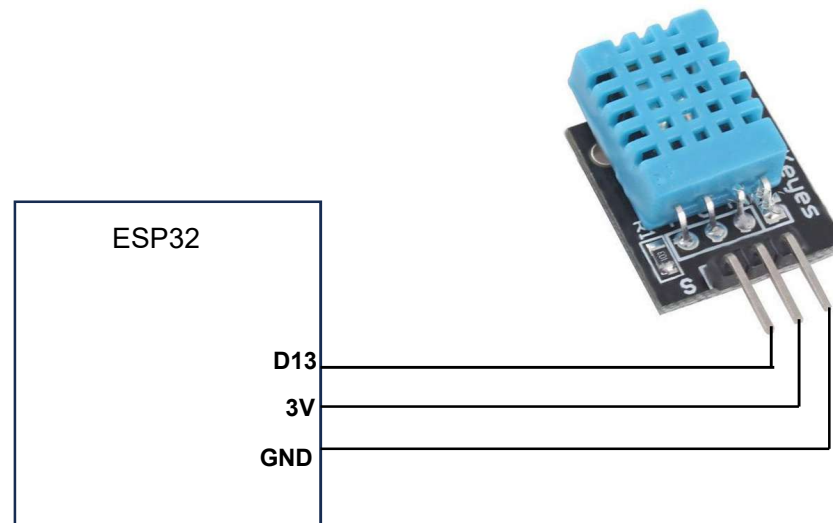
```
#define BLYNK_TEMPLATE_ID "TMPL6E6ndG0Zd"
#define BLYNK_TEMPLATE_NAME "DHT11"
#define BLYNK_AUTH_TOKEN
"rDvI-KSarQTsSs1gU2yaHG_WKCeP800v"
```

Template ID, Template Name, and AuthToken should be declared at the very top of the firmware code.

[Documentation](#) [Copy to clipboard](#)

Sending DHT11 data to Blynk IoT App

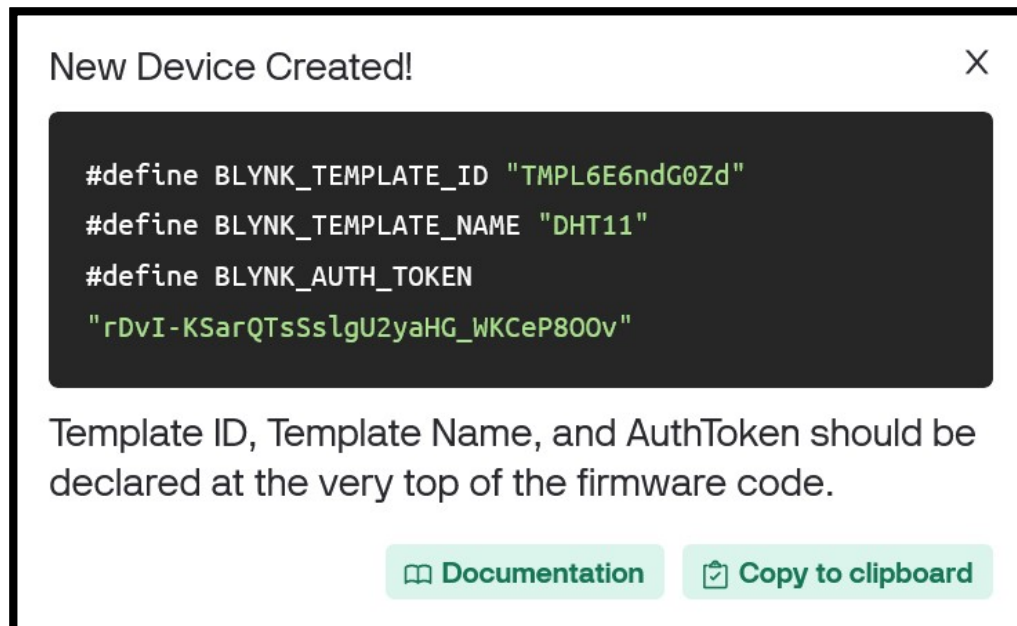
- Build the following circuit on breadboard
- Download **Blynk_DHT11_ESP32.ino** sketch



Sending DHT11 data to Blynk IoT App

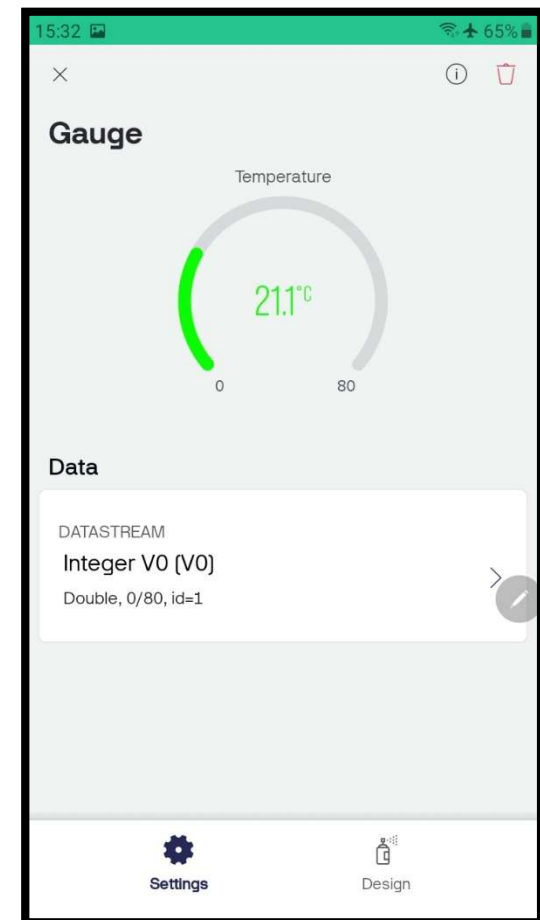
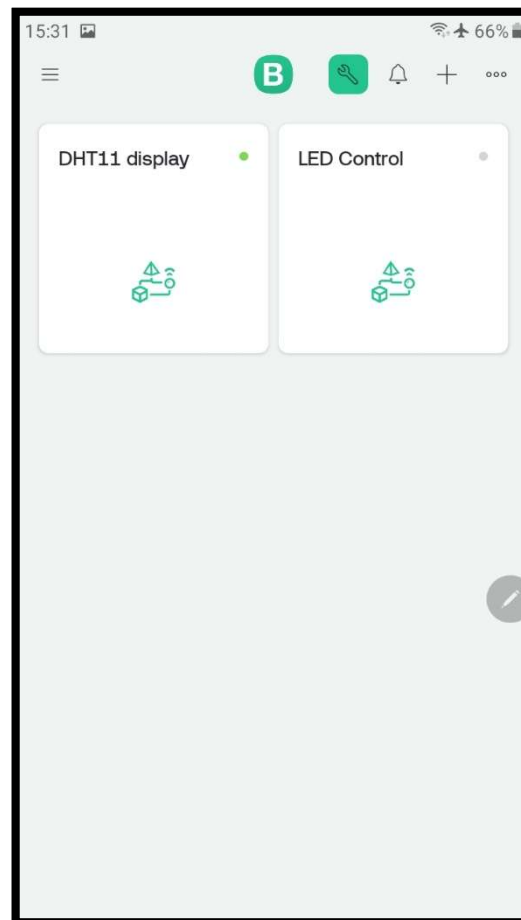
- Add Template ID, Name and AuthToken from Blynk console in your sketch
- Add Wi-Fi AP name and Password, then download sketch to ESP32

```
2 #define BLYNK_TEMPLATE_ID "TMPL6E6ndG0Zd" //<--- ข้อมูลจาก template ที่สร้างขึ้น
3 #define BLYNK_TEMPLATE_NAME "DHT11" //<--- ข้อมูลจาก template ที่สร้างขึ้น
4 #define BLYNK_AUTH_TOKEN "C0IwDPX7YJag-e5KqGUMBd69FXFnqxwK" //<-- ข้อมูลจาก c
```



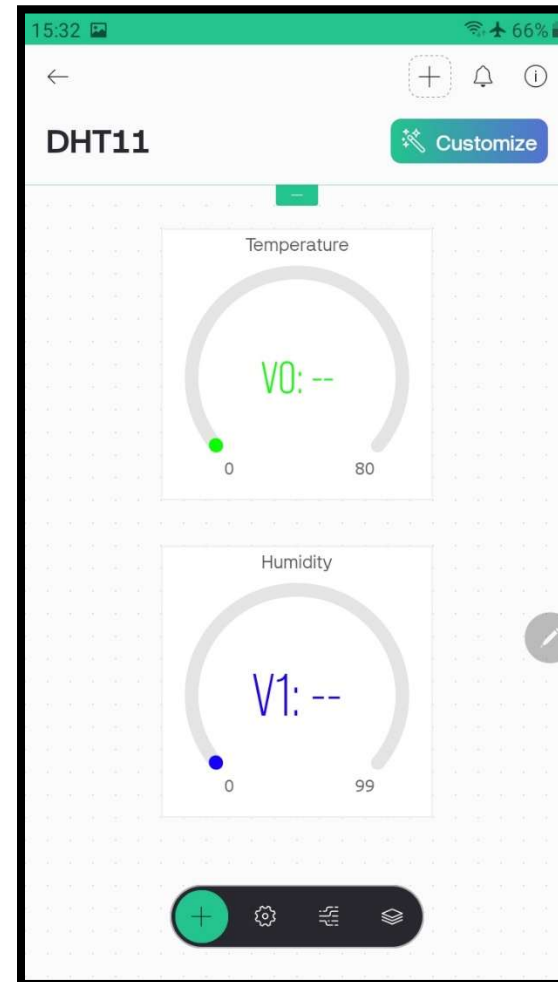
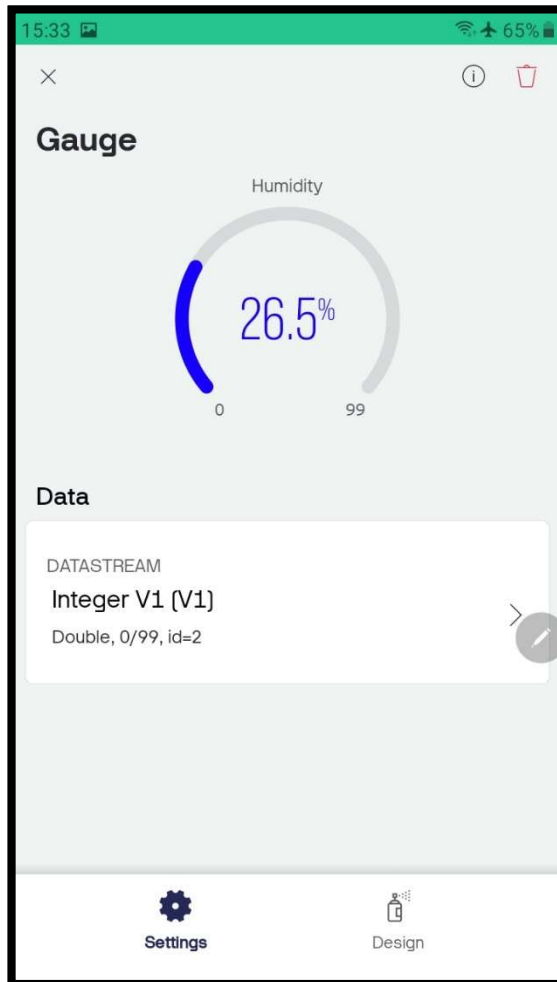
Sending DHT11 data to Blynk IoT App

- Open Blynk IoT App on mobile phone
- Choose **DHT11 Display** template
- Add Gauge Widget and rename to **Temperature** and set DATASTREAM to **Integer V0 [V0]**



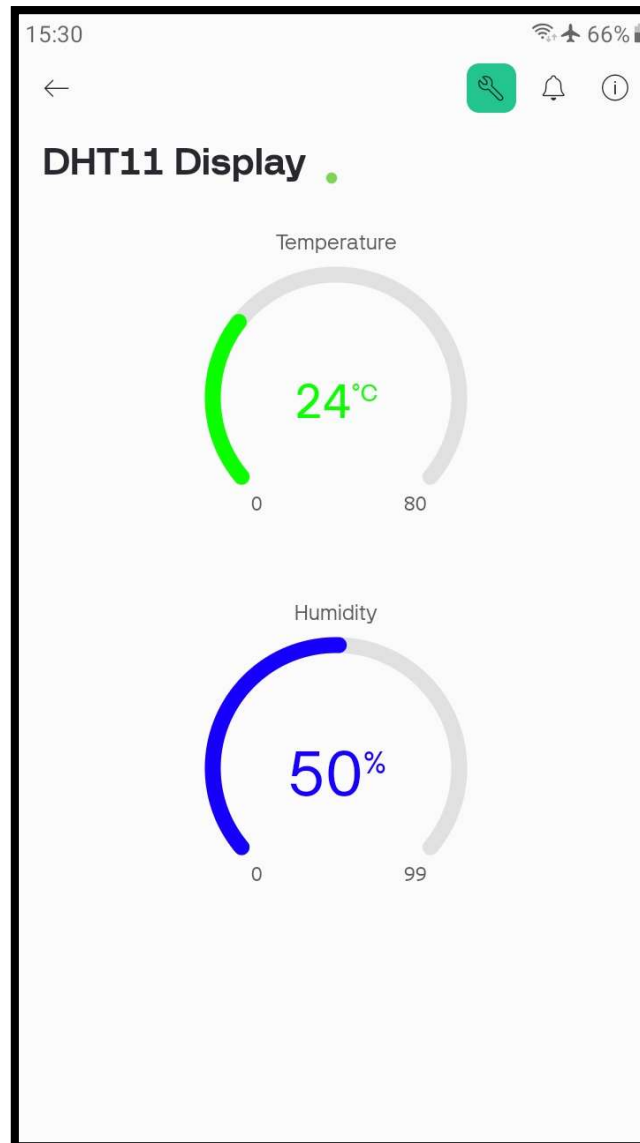
Sending DHT11 data to Blynk IoT App

- Add the 2nd Gauge, rename it to **Humidity**, set DATASTREAM to **Integer V1 [V1]**
- Tap on X to close current screen, then tap on ← to go back to main screen



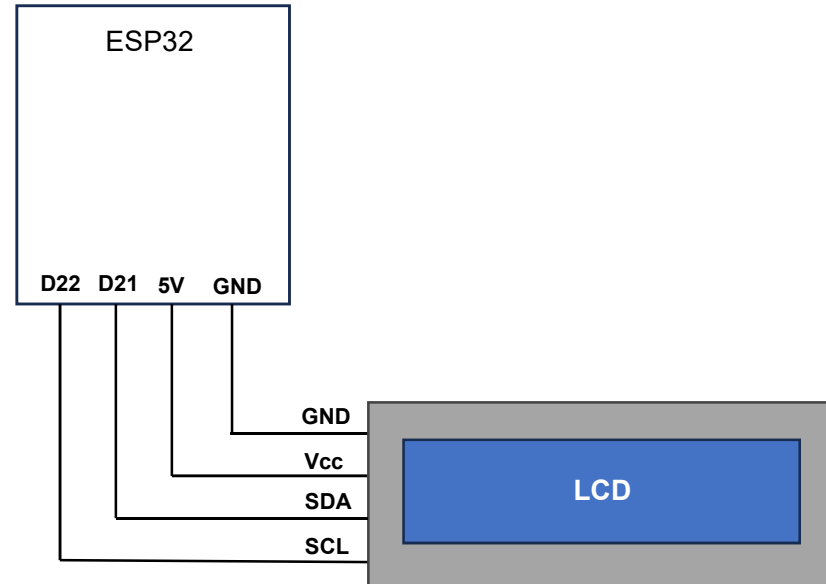
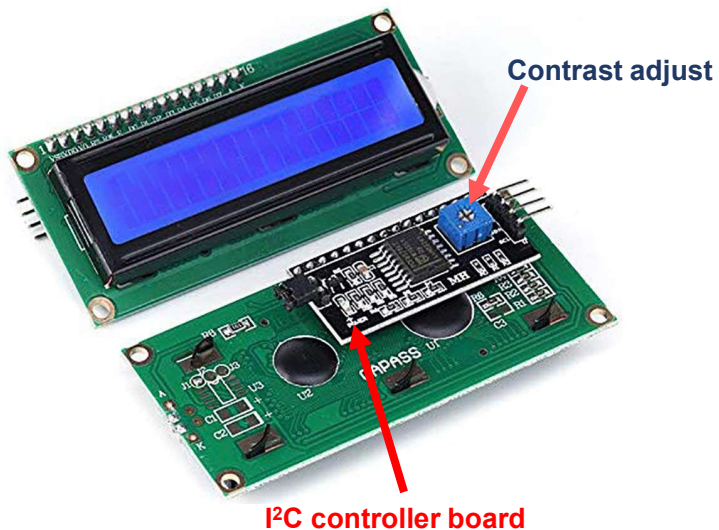
Sending DHT11 data to Blynk IoT App

- Data from DHT11 will be displayed and updated every 5 seconds



Alphanumeric LCD (revisited)

- Inter-Integrated Circuit (IIC or I²C) is used to simplified communication between LCD module and ESP32
- I²C uses only 2 I/O lines for communication



Alphanumeric LCD

- Floating point numbers can be printed with specified number of fraction
- Run test code: **I2C_LCD_Numbers.ino**
- LCD display contain preloaded special characters:

- To print these characters,
use *(char) x* inside lcd.print()

- For example:

```
lcd.print((char) 247); // print  $\pi$ 
```

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	▲	▲	▼	▼	■	■	■		▲	▲	▼	▼	■	■	■	
16																
32		!	"	#	\$	%	&	'	(	)	*	+	,	-	.	/
48	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
64	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
80	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
96	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
112	p	q	r	s	t	u	v	w	x	y	z	{		}	~	€
128																
144																
160		。	「	」	、	・	ヲ	アイ	ウ	エ	オ	カ	キ	ク	ケ	コ
176	ー	ア	イ	ウ	エ	オ	カ	キ	ク	ケ	コ	サ	シ	ス	セ	ソ
192	タ	チ	ツ	テ	ト	ナ	ニ	ヌ	ネ	ノ	ハ	ヒ	フ	ヘ	ホ	マ
208	ミ	ム	メ	モ	ト	ヨ	ラ	リ	ル	レ	ロ	ワ	ン	〃	〃	〃
224	α	ä	ß	ε	μ	σ	ρ	α	Γ	ι	*	φ	τ	ñ	ö	
240	ρ	α	θ	ω	Ω	Ü	Σ	π	Σ	υ	κ	π	π	÷		■

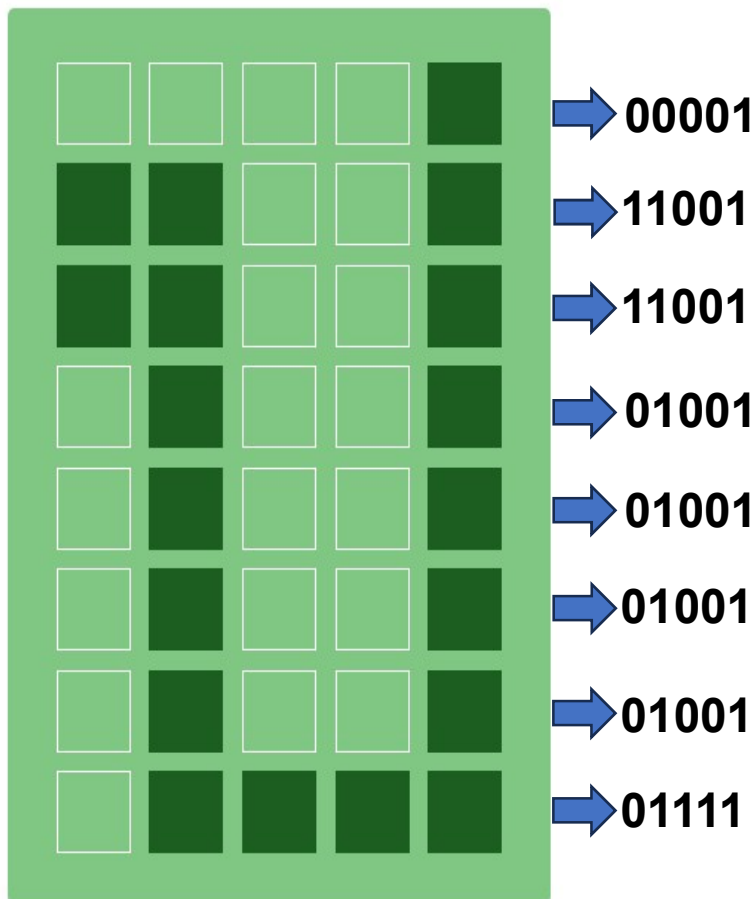
LCD Custom Font Display

- Some applications, we might need special symbols that are not available in LCD character ROM
- It is possible to customize fonts and display them on LCD



LCD Custom Font Display

- Each character in LCD fonts are contained in 5x8 pixel grid
- We consider dark pixel as "1" and light pixel as "0"
- In the example below, we can write binary number for each row as:



```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

LiquidCrystal_I2C lcd(0x3F, 16, 2);

byte customChar[] = {
  B00001, B11001, B11001, B01001,
  B01001, B01001, B01001, B01111
};

void setup() { // Initialize the LCD
  lcd.init();
  for (int i = 0; i < 8; i++) { // Create custom characters
    lcd.createChar(i, thaiChars[i]);
  }
  lcd.clear();
  displayThaiCharacters(); // Display the Thai characters
}
```

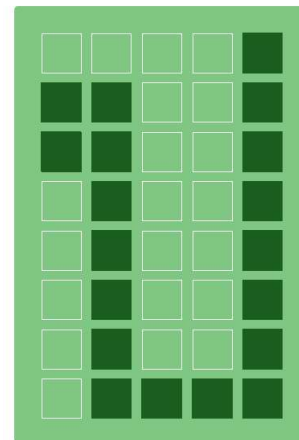
LCD Custom Font Display

- There is an online tool to help design custom fonts:

<https://maxpromer.github.io/LCD-Character-Creator/>

LCD Custom Character Generator

Support character lcd and create code for Arduino.



Clear

Invert

Link

- [Arduino LCD Circuit](#)
- [Arduino LCD I2C Circuit](#)
- [Arduino LCD I2C library](#)

Color ☒ Green ☐ Blue
Microcontroller ☒ Arduino
Interfacing ☐ Parallel ☒ I2C
Data Type ☒ Binary ☐ Hex
Code

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

// Set the LCD address to 0x27 in PCF8574 by NXP
LiquidCrystal_I2C lcd(0x3F, 16, 2);

byte customChar[] = {
  B00001,
  B11001,
  B11001,
  B01001,
  B01001,
  B01001,
  B01001,
  B01001,
  B01001,
  B01111
};

void setup() {
  lcd.begin();
  lcd.createChar(0, customChar);
  lcd.home();
  lcd.write(0);
}

void loop() { }
```

LCD Custom Font Display

- The Liquid_Crystal_I2C library provides function to install custom fonts into available LCD character memory
- **Example: ThaiCharDemo.ino**